Receipt-Exact Measurement of the Atomic-MEV Surface on Post-PBS BNB Smart Chain: The Independent-Searcher Edge Is Realized ≈ Zero

Alessio Rocchi

June 2026

Abstract

Empirical studies of decentralized-exchange (DEX) MEV estimate opportunities analytically from constant-function-market-maker (CFMM) closed forms, which approximate fees, ignore tick-crossing in concentrated-liquidity (V3) pools, and cannot account for reverts, hooks, or rounding—so they systematically over-count, and they price each MEV *category* on a different dataset, making cross-strategy comparison unreliable. We instead **measure two atomic MEV categories — cross-DEX backrun arbitrage and sandwiching — on a single ground-truth instrument and a single victim stream**, by re-executing real BNB Smart Chain (BSC) blocks inside a full node and valuing every opportunity with the **actual EVM**. Our in-process *SimEngine* applies a block's transactions on a copy of the parent state via the node's own `core.ApplyTransaction`, producing receipts that match the canonically stored receipts *exactly*—status, gas, cumulative gas, and every log—validated on real mainnet blocks (5/5 PASS, 100–151 txs each). For backrun we detect cross-venue negative cycles at the correct **intra-block transient** (immediately after each victim swap, the only state where a backrun exists) and value each by the pool's own bytecode—V2 hops by the exact closed form, V3 hops by an in-process PancakeSwap QuoterV2 call on the exact intermediate state—under an optimal-input search and a realistic cost gate. For sandwiching we **construct** the three-leg attack on copied state: a synthetic attacker funded by direct ERC20 storage writes, its frontrun and backrun routed to the victim's *actual* pool (a direct, K-safe `pair.swap` for any V2 fork; the real V3 SwapRouter for Pancake V3), with the *real* victim transaction replayed between, and read the attacker's profit off the EVM denominated in the pool's BNB-priced numeraire.

The two measurements diverge sharply. **Cross-DEX backrun on the deep WBNB/stable hub is economically marginal**: over

3,000 sampled blocks the EVM confirms 803 gross-positive cycles but only 15 **(1.9%) clear the ∼$0.50 gas gate** (280k gas × 3 gwei), ∼$10 total—the deep pools are arbitraged to the fee/gas boundary within each block. **Sandwiching the long tail is not**: over 2,550 sampled blocks we examine 33,525 victim swaps and find 1,735 gross-positive and 1,162 **ex-post net-positive sandwiches at 3 gwei** (median gross $1.78, max $430.50; 1,162 = 67.0% of gross-positive after gas+flash, with 69.0% clearing the 3-gwei gas gate before the flash premium), totalling 35.08 **BNB (∼$20,200)**—entirely on long-tail WBNB-paired pools, not the deep hub. The contrast is the headline: under identical receipt-exact valuation on the same blocks, the independent-searcher atomic-MEV surface on post-PBS BSC is **dominated by sandwiching over backrun by orders of magnitude** (∼90× more net-positive opportunities per block, ∼2,000× more total value), and the surviving edge lives in the long tail, not the deep markets the backrun literature studies. As a coarse external sanity check, our ground-truth ex-post sandwich aggregate extrapolates to a few hundred $M/yr—the same order as loosely-comparable published BSC MEV figures (we are careful that the widely-cited "$289M" is a 2025 cross-context MEV *volume* number, not net BSC sandwich profit, so it is only a rough comparator; realized BSC sandwich profit is smaller). The gap between our ex-post upper bound and smaller realized figures is not noise—it *is* the ex-post-vs-realizable story this paper foregrounds. We then *measure* that story with an **in-block counterfactual**: for each ex-post opportunity we detect whether a real competitor already captured it in the canonical block. Over 2,100 blocks 0 **of** 735 **ex-post sandwich opportunities were realized by any cross-tx sandwich** (capture rate 0.00, with an auditable bracket→same-actor→corroboration funnel and an independent 1,500-block scan confirming cross-tx sandwiches are essentially absent in-window)—realized BSC MEV is atomic-arb-dominated. Crucially, *unrealized is not available*: the surface is empty because cross-tx sandwiching has receded under validator/builder filtering and private order flow (BEP-322; two latency-advantaged builders, 48Club and BlockRazor, see private flow ahead of the public mempool and capture ∼90% of MEV), the same regime that would exclude an independent's sandwich too. A third angle closes the argument: the **censorship-differential** D—the one point-identified, structurally-reachable estimand (the value a builder leaves by *dropping* public, orthogonal, profitable opportunities)—is $\hat{D} = 0$ after a chain-verified settle window, because 99.6% of apparent public "drops" are merely *delayed inclusion*, not censorship. So the independent atomic-searcher edge on post-PBS BSC is, measured from three angles, ≈ 0—backrun sub-gas, the large ex-post sandwich surface realized by no one, and the public residual empty. The transferable contribution is a validated full-EVM instrument that prices

atomic categories on one substrate—receipt-exact, more faithful than analytic CFMM for V3—the first apples-to-apples (to our knowledge) backrun-vs-sandwich measurement on one instrument (complementing per-target executing optimizers such as Lanturn [5] with a per-block, full-node census), an in-block counterfactual that converts "ex-post existence" into a measured capture rate, and a settle-windowed censorship-differential. During development we found and fixed four implementation bugs in the detectors, documented in the methods for reproducibility.

Contributions.

1. A receipt-exact, in-process simulation methodology (SimEngine) that re-executes real blocks on copied node state via `core.ApplyTransaction` and matches stored receipts byte-for-byte (5/5 PASS)—ground-truth execution, strictly more faithful than analytic CFMM for V3 / fee-on-transfer / hooks / reverts—used as the *single* valuation substrate for *both* atomic MEV categories we measure.

2. A full-EVM ground-truth valuation oracle for **backrun** (Stage B): every candidate cross-venue cycle valued by the actual pool/Quoter bytecode (V2 closed form; V3 via in-process QuoterV2 on the exact intermediate state) with optimal-input search, evaluated at the correct intra-block transient (post-block evaluation is *not* a backrun test—intra-block competitors re-align prices).

3. A full-EVM ground-truth **sandwich** constructor: a synthetic attacker funded by direct storage writes (hardcoded + runtime-probed ERC20 slots), frontrun/backrun routed to the victim's actual pool (K-safe direct `pair.swap` for any V2 fork; the real V3 SwapRouter for Pancake V3), the real victim tx replayed between, profit read off the EVM in a single BNB-priced numeraire—the first (to our knowledge) receipt-exact *per-block census* of the sandwich surface (complementing per-target executing optimizers such as Lanturn [5], which construct and EVM-simulate sandwich/arbitrage strategies for a chosen contract rather than censusing every victim swap in a block).

4. **The first apples-to-apples backrun-vs-sandwich measurement on one instrument and one victim stream**: same blocks, same EVM valuation, same cost model—turning a cross-paper comparison into a single controlled experiment. Prior executing tools (e.g. Lanturn [5]) EVM-evaluate constructed strategies but optimize one target at a

time; we instead hold a single per-block victim stream fixed and price *both* atomic categories on it.

5. A measurement of the independent-searcher atomic-MEV surface: backrun is frequent but **sub-gas** (15 of 803 net-positive over 3,000 blocks, ∼$10); sandwiching the long tail is **substantial** (1,162 of 1,735 net-positive over 2,550 blocks, 35.08 BNB ≈ $20,200)—sandwiching dominates by ∼90× in count and ∼2,000× in value, and the surviving edge is a long-tail phenomenon, not a deep-pool one.

6. A coarse external sanity check: the ground-truth ex-post sandwich aggregate extrapolates to a few hundred $M/yr, the same order as loosely-comparable published BSC MEV figures—with the explicit caveat that the widely-cited "$289M" is a cross-context MEV *volume* number, not net BSC sandwich profit, and that the gap to smaller *realized* figures is itself the ex-post-vs-realizable result.

7. **A ground-truth in-block counterfactual** that converts ex-post existence into a *measured* realizability: for each ex-post opportunity it detects whether a real competitor already captured it in the canonical block (conjunctive bracket + same-actor + profit-corroboration gates, conservative by construction, with an auditable funnel). Over 2,100 blocks it finds 0 **of** 735 **ex-post sandwiches realized by any cross-tx sandwich** (capture rate 0.00); cross-referenced with a 1,500-block pool-agnostic scan, cross-tx sandwiching is essentially absent in-window and realized MEV is atomic-arb-dominated—so the independent atomic edge is ≈ 0 by direct measurement, not just by argument.

8. A careful realizability / threats discussion separating ex-post existence (what we prove) from realizable capture (which we now measure), uniform across both strategies, including the *unrealized* ≠ *available* point, with an explicit statement of what would change the conclusion.

1 Introduction

1.1 Maximal Extractable Value and the measurement problem

Maximal Extractable Value (MEV)—the profit a party can extract by including, excluding, or reordering transactions within the blocks it produces

or influences—has grown into a structural feature of every major smart-contract chain since Daian et al. introduced the concept and documented priority-gas auctions and consensus-layer instability in *Flash Boys 2.0* [10]. A large share of MEV is *atomic arbitrage*: a single transaction that buys cheaply on one DEX and sells dearly on another. When it trails the user swap that *created* the discrepancy, it is a *backrun*.

The empirical literature—cyclic arbitrage [21], quantifying blockchain extractable value [17], front-running [19], large-scale measurement [16]— shares a rarely-examined choice: **opportunities are estimated analytically**. A study reads reserves, applies a CFMM closed form (usually Uniswap V2 $x \cdot y = k$), and reports the implied trade. This embeds approximations that distort the answer: it assumes $x \cdot y = k$ even for pools that do not obey it (concentrated-liquidity V3 [1], stableswap); it cannot see *reverts*; it ignores *fee-on-transfer tokens, hooks, and rounding*; and it conflates ex-post with realizable opportunity. The net effect is that analytic studies **overcount**. Even at the detection layer, Zhang et al. show a naive negative-cycle scan surfaces a fraction of what a better graph transform finds [22]; at the valuation layer the gap to exact, fee-and-tick-accurate profit is larger still.

1.2 Our instrument: receipt-exact in-process re-execution

Rather than approximate execution, **we execute**. We run a full BSC node (bsc-geth v1.7.3, snapshot-seeded, 64-core / 125 GB host) and build an in-process *SimEngine* that applies a block's transactions on a *copy* of the parent state through the node's own `core.ApplyTransaction`—the identical path the canonical processor uses. The canonical state is never mutated.

The SimEngine is **validated against ground truth**: we replay real mainnet blocks and compare simulated receipts to the stored receipts. They match *exactly*—status, gas used, cumulative gas used, and every log's address, topics, and data—5/5 PASS on blocks around height 101.83M with 100–151 txs each. This *is* execution, with a receipt-level certificate. For V3 this is decisive: rather than re-implement `TickMath`/`SwapMath`/`SqrtPriceMath` we let the pool bytecode be its own arbiter [1, 12].

1.3 The post-PBS BSC regime, and why this result is interesting

BSC is high-throughput, deeply liquid in a small hub (WBNB, USDT, USDC, USD1, BUSD), and recently underwent a proposer–builder separation (PBS) transition via BEP-322 [7]: no relay, whitelisted builders, private order flow

ahead of the public mempool. Wang et al. [20] report that two builders (48Club, BlockRazor) produce most blocks and capture most MEV; the dominant strategy is short (2–3 swap) cycles on the WBNB/stablecoin hub; opportunities live ∼100–400 ms; public-mempool transactions reach external searchers tens of ms *after* builder bundles.

This is the backdrop against which we ask, more broadly than the backrun literature does, **what the atomic-MEV opportunity surface available to an *independent* searcher actually looks like on post-PBS BSC**—across both of the atomic categories a non-builder could in principle contest. We split it into two questions. **RQ1: is there any net-positive independent-searcher *backrun* edge left on BSC's major pools?** And **RQ2: how does that compare, on the *same* instrument and the *same* victim stream, to *sandwiching*—the dominant atomic MEV category on BSC by value (roughly half of measured BSC MEV)?** Treating both on one ground-truth substrate is the methodological move that makes the comparison meaningful, because the existing literature prices each category on a different dataset with a different (analytic) model.

For RQ1, our naive baseline (three deep Pancake V2 pools, exact closed-form optimal input, 250k-gas / 3-gwei gate, zero builder bid) finds **zero profitable backruns over** 5,100 **blocks**; a cross-DEX V2 graph extension finds **zero candidates over** 2,950 **blocks**. *But they measure the wrong thing*: both evaluate against *post-block* reserves, where cross-venue prices are already re-aligned by intra-block competitors. Backrun MEV lives in the transient state immediately *after* a victim swap; end-of-block evaluation misses it. These post-block zeros quantify *standing* arbitrage, not backrunable transients—a realization that reframes the study.

1.4 The correct experiment: intra-block, EVM-valued

The SimEngine re-executes each block transaction-by-transaction, so we hook in **after every transaction that emits a watched-pool** `Swap`, snapshot reserves there, and detect+value at the only state where a backrun could exist. The final model: (1) builds a token graph over a verified 12-pool multi-DEX set (six V2: three Pancake, three Biswap; six Pancake V3 across tiers 100/500/2500), edges weighted $-\ln(r_{uv}(1-\text{fee}))$; (2) runs negative-cycle detection (Stage A, *detection only*, marginal prices) for cycles of length $2..K$ from WBNB; (3) **values every candidate with the EVM (Stage B), the sole oracle**: V2 hops by the closed form, V3 hops by an in-process PancakeSwap *QuoterV2* call on the exact intermediate `state.Copy()`, with optimal input by golden-section over EVM probes; (4)

applies net = gross $-\sum$ gas $-$ bid with measured per-cycle gas ($\sim$280k) at 3 gwei ($\approx$\$0.50), assuming capital-free flash funding [18], and emits a gross-profit distribution, break-even gas price, and a $\{0, 0.1, 0.3, 1, 3\}$ gwei sweep. **RQ3** follows: the gross-positive$\rightarrow$net-positive collapse measures the analytic over-count.

The same intra-block victim stream answers **RQ2** with no change of instrument. For *every* watched-pool victim swap the SimEngine surfaces, we do not merely look for a trailing arbitrage—we **construct the adversarial sandwich**: on a fresh copy of the pre-swap state we fund a synthetic attacker, insert an optimally-sized frontrun *in the victim's own pool*, replay the *real* victim transaction against the degraded reserves, close with a backrun, and read the attacker's profit straight off the EVM (§3.7). Because the construction routes to the victim's actual pool by a K-safe direct `pair.swap`, it generalizes beyond the deep hub to **any V2-fork pool in the long tail**—exactly where, our own backrun runs hint, the volume actually is (the deep pools were quiet). This lifts the study from "is backrun profitable on the hub?" to "which atomic strategy, backrun or sandwich, carries the independent-searcher edge, and where?"—and the answer (RQ2) turns out to dominate the answer to RQ1 by orders of magnitude.

1.5 Summary of findings

- The SimEngine reproduces real mainnet receipts exactly (**5/5 PASS**, 100–151 txs each): *one* ground-truth execution instrument, used for *both* strategies.

- **Backrun (deep hub).** The naive deep-pool model finds 0/5,100 **blocks** and the cross-DEX V2 graph 0/2,950 **blocks** (both *post-block*, not valid backrun tests). The correct intra-block detector + EVM oracle, over 3,000 **sampled blocks**, confirms 803 **ground-truth gross-positive cross-venue cycles ($\sim$0.27 per block), of which only** 15 (1.9%) **are ex-post net-positive** at 3 gwei—$\sim$1 per 200 sampled blocks, 0.0172 BNB ($\sim$\$10) total; the other 788 sit below the $\sim$\$0.50 gas floor. Frequent, real, but **overwhelmingly sub-gas**.

- **Sandwich (long tail).** The EVM-constructed three-leg attack, over 2,550 **sampled blocks** examining 33,525 **victim swaps**, finds 1,735 **gross-positive and** 1,162 **ex-post net-positive sandwiches** at 3 gwei ($\sim$0.46 per block), totalling 35.08 **BNB ($\sim$\$20,200)**; median gross \$1.78, p90 \$31.6, max \$430.50. The 1,162 net-positive = 67.0% of

gross-positive after gas+flash; 69.0% (1,197) clear the 3-gwei gas gate before the flash premium—entirely on long-tail WBNB-paired pools.

- **The contrast (headline).** Same instrument, same victim stream: sandwiching yields **$\sim$90$\times$ more net-positive opportunities per block and $\sim$2,000$\times$ more total value** than backrun. The independent searcher's atomic-MEV edge is a **long-tail sandwiching phenomenon, not a deep-pool backrun one**.

- **External sanity check (with caveat).** The ground-truth *ex-post* sandwich aggregate extrapolates to a few hundred \$M/yr—the same order as loosely-comparable published BSC MEV figures. We flag that the widely-cited "\$289M" is a cross-context MEV *volume* number, not net BSC sandwich profit, so it is only a coarse comparator; *realized* BSC sandwich profit is smaller, and that gap is the ex-post-vs-realizable finding (§5.8.1), not a flaw in the measurement.

- **Realizability (in-block counterfactual).** Of the **735 ex-post sandwich opportunities** in a 2,100-block window, **0 were actually captured** by a real cross-tx sandwich (capture rate **0.00**), with an auditable funnel (1,077 brackets $\rightarrow$ 79 same-actor $\rightarrow$ 0 corroborated) and an independent 1,500-block scan confirming cross-tx sandwiches are essentially absent in-window—realized MEV is atomic-arb-dominated. *Unrealized* $\neq$ *available*: the surface is empty because sandwiching is suppressed for everyone (filtering + private flow), which excludes an independent identically.

- **Censorship-differential (the reachable estimand).** The one point-identified, structurally-reachable estimand—the value a builder leaves by *dropping* public, orthogonal, profitable opportunities—is $\hat{D} = 0$ after a chain-verified settle window; 99.6% of apparent public "drops" were merely *delayed inclusion* (mined a few blocks later), not censorship. The public residual is empty too.

- **Implementation bugs (reproducibility).** During development we found and fixed four implementation bugs in the detectors (a units bug; a realizability false-zero; a censorship estimand-inversion; a censorship delayed-vs-censored conflation), each documented in the methods so the results are reproducible.

The conclusion: on post-PBS BSC the *ex-post* atomic-MEV surface looks like it has **migrated from deep-pool backrun (15 of 803 net-positive,**

∼$10) to long-tail sandwiching (1,162 net-positive, ∼$20,200)—but *realized*, neither is capturable by an independent. Backrun is sub-gas; the long-tail sandwich surface, though large ex-post, is realized by **no one** (capture rate 0 over 2,100 blocks) and would be filtered for us too. Under BEP-322 two latency-advantaged builders (48Club, BlockRazor) produce most blocks, see private flow ahead of the public mempool, and capture ∼90% of MEV [7, 20]. So the independent atomic-searcher edge is, by *direct measurement*, ≈ 0. The transferable contribution is the *instrument and protocol*: validated full-EVM ground-truth valuation that prices every atomic category on one substrate, evaluated at the correct intra-block transient (backrun gross→net collapse $803 \to 15$—the quantitative case against analytic CFMM over-counting, especially for V3), the first controlled backrun-vs-sandwich contrast on a single per-block victim stream (to our knowledge; complementing per-target executing optimizers such as Lanturn [5]), and an in-block counterfactual that turns "ex-post existence" into a measured realizability.

2 Related Work

We organize prior work into seven threads: (i) MEV foundations, (ii) cyclic / graph-based arbitrage detection, (iii) convex-optimization routing and optimal sizing over CFMMs, (iv) concentrated liquidity, (v) flash loans / capital-free execution, (vi) sandwich attacks and their concentrated-liquidity profitability, and (vii) the BSC PBS regime that frames the interpretation.

2.1 MEV foundations

Flash Boys 2.0 [10] introduced MEV, priority-gas auctions, and time-bandit attacks. Heimbach and Wattenhofer's SoK [15] systematizes reordering manipulations and defenses (private flow, fair ordering, encrypted mempools) the BSC PBS regime partially instantiates. Qin et al. [17] quantify blockchain extractable value and the consensus-security incentive to fork; Torres et al. [19] analyze ∼11M blocks for ∼200k front-running attacks. *Position:* these works motivate *why* arbitrage MEV is worth measuring; we contribute a measurement *instrument*—receipt-exact realized value rather than analytic inference. The closest non-analytic prior art is Lanturn [5], whose adaptive-learning optimizer constructs MEV-extracting transaction sequences and *executes them on real chain state without modification* (over Sushiswap/Uniswap V2/V3/Aave), valuing sandwich, arbitrage, and backrun strategies by simulation rather than closed form. Lanturn optimizes value for a *chosen target contract*; we instead run a *per-block, full-node census* that prices every

watched victim swap in every block. Our instrument and Lanturn are thus complementary: per-target optimizer vs. per-block opportunity census, both refusing the analytic CFMM surrogate.

2.2 Cyclic and graph-based detection

The standard technique adapts FX negative-cycle search: tokens as nodes, ordered pool-directions as edges of weight $-\ln(r_{uv}(1-\text{fee}))$; a profitable cycle is a negative cycle, found by Bellman–Ford ($O(VE)$), SPFA, or Johnson's. Wang et al. [21] give the empirical framing (>$138M cyclic-arb revenue). Known limits for profit: marginal-rate weights certify only an infinitesimal trade; vanilla Bellman–Ford returns one arbitrary cycle; the signal is a percentage. Zhang et al. [22] address these with a line-graph + Modified MBF pass; McLaughlin et al. [16] are the large-scale precedent motivating detect-then-verify. *Position:* we use negative-cycle detection strictly as a candidate generator, never the profit oracle.

2.3 Convex routing and optimal sizing

For a two-pool $x \cdot y = k$ cycle the optimal input is a closed form (collapse two hops, solve $dP/dx = 0$), implemented in `strategy/arb.go` with integer floor-sqrt. For multi-hop/split routing, convex optimization over CFMMs is the principled framework: concavity of trading functions [3], multi-asset convex programs [2], the canonical routing program where arbitrage is the special case [4], and the dual decomposition into per-market arbitrage subproblems [12], generalized to convex flows over hypergraphs [11]. *Position:* closed form for isolated V2 cycles, dual-decomposition for coupled/split, and for V3 we let the EVM size the trade (golden-section over Quoter evaluations), exact where no closed form exists.

2.4 Concentrated liquidity (V3)

Uniswap v3 Core [1] bounds liquidity to price ranges; $x \cdot y = k$ breaks because L is piecewise-constant across ticks, and exact output is a tick-by-tick iteration with no single closed-form optimum. *Position:* we use `slot0.sqrtPriceX96` only as a candidate detector and *never* compute V3 profit analytically—we defer to the EVM (QuoterV2 / pool bytecode), strictly more accurate than any re-implementation and already receipt-validated.

2.5 Flash loans and capital-free execution

Qin et al. [18] formalize flash-loan search over chain state. Flash swaps / flash loans reduce required inventory to ~ 0, so the binding constraints become gas + builder bid (plus a flash premium for Aave). *Position:* we assume flash funding, net = gross − gas − bid(−flashPremium); V2 in-kind premium is 0 (our default).

2.6 Sandwich attacks: formalization and concentrated-liquidity profitability

Where cyclic/backrun arbitrage exploits a *pre-existing* cross-venue discrepancy, a **sandwich** *manufactures* the discrepancy by front- and back-running a victim swap in the same pool. Zhou et al., *High-Frequency Trading on Decentralized On-Chain Exchanges* [23], were the first to formalize and quantify sandwiching, deriving the optimal frontrun size for a constant-product pool and showing a single adversary could earn thousands of USD/day on Uniswap. Qin et al. [17] subsequently folded sandwiching into their chain-wide extractable-value census alongside arbitrage and liquidations, and the front-running measurement of Torres et al. [19] situates it in the broader transaction-ordering threat surface systematized by Heimbach and Wattenhofer [15].

The constant-product optimal-frontrun closed form does not survive contact with concentrated liquidity. Gogol et al. [13] develop an attacker-profitability model for both CPMMs and concentrated-liquidity AMMs (Uniswap/Pancake v3) and derive the corresponding optimal strategy, confirming that the V3 sandwich optimum is not a single closed form because effective liquidity is piecewise-constant across ticks—the same obstacle we hit for V3 *backrun* sizing and resolve the same way: we use the closed form only to *seed* a search whose every probe is a ground-truth EVM evaluation of the full three-leg construction (§3.7), so V3 sandwich profit is priced by the pool's own `TickMath`/`SwapMath` rather than any analytic surrogate. *Position:* sandwiching is the dominant atomic MEV category on BSC by value—roughly half of measured BSC MEV—yet the empirical-measurement literature prices it analytically or by log heuristics (e.g. the large-scale post-Merge Ethereum remeasurement of Chi et al. [9], ∼3M sandwiches detected by pattern rules rather than re-execution), inheriting the V3 and revert mis-pricing we set out to eliminate; the one close non-analytic precedent, Lanturn [5], EVM-simulates sandwiches but per chosen target, not as a per-block census. We contribute the first (to our knowledge) *receipt-exact, EVM-constructed* per-

block census of the sandwich opportunity surface available to an independent searcher on post-PBS BSC, on the **same instrument and the same victim stream** as our backrun measurement—making the two directly comparable and turning "which atomic strategy actually has an edge?" from an apples-to-oranges literature comparison into a single controlled experiment.

2.7 The BSC PBS regime (BEP-322)

BSC adopted PBS via BEP-322 [7]: Builder API, no relay, whitelisted builders, private flow ahead of the mempool. Wang et al. [20] measure that two builders produce most blocks and capture most MEV, the dominant strategy is short hub cycles, opportunities live ∼100–400 ms, and path length barely correlates with profit. *Position:* this explains our null on the major hub pools and points residual edge (if any) to longer/cross-DEX/V3 and long-tail cycles—scoped in §6.

2.8 Summary of what we adopt vs. scope out

3 Methodology

3.1 Node substrate

A single bsc-geth v1.7.3 full node (PBSS state, snapshot-seeded, 64-core/125 GB), synced to BSC mainnet (chainId = 56, head ∼block 102.54M). Stock v1.7.3 plus read-only instrumentation (`simengine`, `strategy`, a disarmed Phase-3 `builder`/`wallet`/`contracts`); built with `GOTOOLCHAIN=auto` only to `/tmp/geth-sim10`. Every on-chain constant—addresses, V2 slot 8 packing, V3 `slot0` layout, fee tiers, decimals, reserves, selectors—was verified directly against this node, the same state source the SimEngine executes on.

3.2 The SimEngine

`simengine/simengine.go` speculatively executes a tx list on a copy of canonical state, mirroring `core.StateProcessor.Process` and the miner worker but never committing: switch to `statedb.Copy()`; apply system-contract upgrades keyed off *parent* block time; seed the gas pool from `header.GasLimit` minus Parlia reserved gas; build the EVM with the header coinbase as author; run pre-execution system calls; iterate txs, *skipping Parlia system transactions* (they run in `Finalize`), with per-tx snapshot/revert on error; return receipts, flattened execution-ordered logs, gas, and balance deltas. The single loop `applyOnState` is shared by the self-test and the backtest, so validation

and measurement are byte-for-byte the same code. Running the actual pool bytecode handles V3 tick-crossing, stableswap, fee-on-transfer, hooks, slippage reverts, and rounding exactly—an analytic study must model each, and any divergence is a silent error; we have none by construction.

3.3 Validation against stored receipts (5/5 PASS)

`simengine/selftest.go` replays real blocks and compares simulated receipts to the stored receipts by tx hash over **Status, GasUsed, CumulativeGasUsed** and every log's **Address, Topics, Data**. Result: **5/5 PASS** on blocks around height 101.83M, 100–151 txs each, no field mismatch on any tx (Table 2). Matching every log and gas figure across 100+ txs on copied state is an extremely tight constraint that implicitly exercises the post-state root.

3.4 Backtest protocol

`simengine/dryrun.go` is read-only and crash-safe (`defer/recover`; no-op unless `SIMENGINE_DRYRUN` is set; never submits). Per imported head it re-executes the whole block on a `state.Copy()` of the parent. Three modes: *backtest* and *graph* (post-block, v1/v2; evaluate the boundary—not a valid backrun test, §5.3); *intrablock* (v3/v4; the correct test) hooks in after each watched-pool `Swap`, snapshots reserves there, builds the graph, enumerates cycles $2..K$ from WBNB, and values each (V2 closed form; V3 via the EVM oracle). Funnel counters and the distribution accumulator are tallied per processed block and logged on the `TallyEvery` cadence. Reserves: V2 slot 8 holds packed $(\text{reserve0}, \text{reserve1}, \text{ts})$; V3 `slot0` holds `sqrtPriceX96` (low 160 bits) and `tick`, spot $P = (\text{sqrtPriceX96}/2^{96})^2$. A disarmed Phase-3 submitter exists only to make the realizability discussion concrete; it is not used for any measurement.

3.5 The EVM valuation oracle (Stage B)

For each candidate the oracle chains hop outputs into hop inputs on the exact intermediate `state.Copy()`: V2 hops by the exact closed form (which the self-test validates to receipt level for real V2 swaps); V3 hops by a read-only, `Snapshot`/`Revert`-bracketed EVM call (`simengine/evmcall.go`, `EthCall`) to PancakeSwap *QuoterV2* (`0xB048Bbc1...25997`, selector `0xc6a5026a`), which reverts-to-return the exact output the pool would produce. Optimal input is found by golden-section (`strategy/quoter_oracle.go`) whose every probe is an EVM evaluation; per-cycle gas is `CycleGasUnits(cycle)`

(∼280k). net = gross − `CycleGasUnits` · gasPrice − bid − margin. This is the *sole* profit authority; no profit is ever taken from Stage A.

3.6 Metrics

Coverage (blocks processed, sampling fraction); the candidate funnel (Stage-A candidates, EVM gross-positive, net-positive); the gross-profit distribution (USD percentiles + exact max); the break-even gas-price distribution (gwei); a $\{0, 0.1, 0.3, 1, 3\}$ gwei sensitivity sweep; and the structural breakdown by DEX mix and cycle length. The Stage-A→Stage-B and gross→net ratios give RQ3. For sandwiching (§3.7) we report the analogous funnel—victims seen, fundable, supported, above-threshold, gross-positive, net-positive—plus the same gross-profit distribution, break-even gas, and gas-sweep, all denominated in BNB.

3.7 Sandwiching: ground-truth construction and valuation

Backrun arbitrage is *parasite-free*: the searcher inserts one transaction after a victim swap and the victim is unaffected. A **sandwich** is adversarial and three-legged: the attacker (1) **frontruns**—buys the same direction as the victim, worsening the victim's execution price; (2) lets the **victim** execute at the degraded price; (3) **backruns**—sells the position back into the pool the victim just pushed, realizing the spread the attacker manufactured. Unlike backrun arbitrage, sandwich profit is *created* by the attacker's own frontrun, so it cannot be read off pool reserves analytically without re-deriving the full three-trade price path—and on real pools that path includes V3 tick-crossing, fee-on-transfer tokens, and reverts that closed-form CFMM math silently mis-prices [23, 13]. This is exactly where the SimEngine's receipt-exact substrate pays off a second time: we *construct* the sandwich and let the EVM price it.

3.7.1 The construction

For each victim swap surfaced by the intra-block trigger (§4.5) we build a synthetic 3-tx sequence on a fresh `parent.Copy()` and re-execute it with the same `core.ApplyTransaction` / in-process `vm.Call` machinery validated in §3.3:

1. **Fund a synthetic attacker.** A throwaway attacker address is given inventory of the pool's *numeraire* token (§3.7.3) by writing the ERC20 balance and allowance storage slots directly on the copied state—never

on canonical state. For the hub tokens the slots are known and hard-coded (WBNB balance slot 3 / allowance slot 4; USDT, USDC balance slot 1 / allowance slot 2). For arbitrary long-tail tokens we **probe the layout at runtime**: we write a sentinel to candidate `keccak256(addr . slot)` mappings for slot $\in \{0..12\}$, call `balanceOf(addr)` through the EVM, and accept the first slot whose read returns the sentinel (cached per token). Tokens whose `balanceOf` cannot be reproduced by any standard mapping slot (proxies, exotic layouts) are counted `skippedUnfundable` and excluded—a *conservative* exclusion that can only *lower* the measured sandwich count.

2. **Frontrun.** The attacker swaps an optimally-sized amount of numeraire into the victim's direction, *on the victim's actual pool.* Routing matters: we do not assume a fixed router. V2-family pools (Pancake V2 and any $x \cdot y = k$ fork—Biswap, ApeSwap, ...) are swapped by encoding a **direct** `pair.swap(amount0Out, amount1Out, to, data)` (selector `0x022c0d9f`) with a K-safe `amountOut`: we compute the output with the *Pancake* 0.25% fee even when the fork's true fee is lower, which *under-quotes* the attacker's output and so never over-states sandwich profit on a fork we have not fee-calibrated. Pancake V3 pools are swapped through the real Pancake V3 `SwapRouter` (`0x1b81D678...213eB14`, `exactInputSingle` with the deadline-bearing selector `0x414bf389`). Non-Pancake V3 / Algebra pools we do not yet route and count `skippedUnsupported`.

3. **Victim.** The victim's *real* transaction is replayed verbatim on the post-frontrun state (no snapshot/revert bracketing—§3.7.4), so the victim executes against exactly the reserves the attacker's frontrun left behind, with the EVM applying every slippage guard the victim's own calldata carries (a sandwich that would trip the victim's `amountOutMin` simply yields a revert and zero profit, caught for free).

4. **Backrun.** The attacker swaps its entire acquired position back through the same pool, again via the K-safe direct path, closing the loop.

The attacker's **gross** is the change in its numeraire balance across the three legs, read from the copied `StateDB`—the EVM's own number, inclusive of all three real swaps' fees, tick crossings, and rounding. No CFMM closed form is used to *value* the sandwich; the closed form (§4.5, `strategy/sandwich.go`) is used *only* to seed the optimal-frontrun search.

3.7.2 Optimal frontrun size

The frontrun size trades off manufactured spread against the attacker's own price impact: too small and the victim is barely moved, too large and the attacker pays more slippage on entry/exit than it extracts. We seed with the V2 closed-form sandwich optimum (γ-parameterized, `strategy/sandwich.go`) and refine by evaluating candidate sizes through the full EVM construction above, capping the frontrun at $\leq$100% of the victim's input notional and $\leq$50% of the pool's numeraire reserve (a liquidity-sanity bound that prevents the optimizer from proposing a frontrun the pool could not absorb without absurd impact). The reported gross is the EVM value at the best feasible size.

3.7.3 Denomination in a single numeraire (and an integrity catch)

A sandwich's three legs move two tokens; profit must be reported in **one** unit. Our first any-pool implementation contained a **units bug** worth recording because it shaped the protocol: it measured the attacker's gross in whatever token the victim happened to *spend* (often an arbitrary memecoin) while subtracting the gas cost in BNB. Mixing a memecoin-denominated gross with a BNB-denominated cost made the net gate meaningless and produced absurd aggregates (a total "net" of order 10^{30} wei—$\sim 10^{12}$ BNB—and an implied break-even gas price of $\sim 10^{9}$ gwei). The figures were *flagged as not-real on sight* and never reported; the bug was the detector's, not the chain's.

The fix defines, for each victim pool, a **numeraire**—the side that is a hub asset with a known BNB price (WBNB directly; USDT/USDC via the WBNB/stable pool). The synthetic attacker is funded *in the numeraire*, both swap legs are denominated in the numeraire, and gross/net/gas/flash-fee are all expressed in BNB. Pools with **no** hub-asset side (pure token/token pairs) cannot be denominated and are counted `skippedNoNumeraire` and excluded. This both fixes the accounting and scopes the measurement to economically interpretable opportunities. After the fix the same sample produced realistic magnitudes (per-sandwich net of order 10^{-4}–10^{-2} BNB, aggregate tens of BNB, break-even gas in the single-to-hundreds-of-gwei range) —sanity-checked against the published BSC sandwich-MEV figure before being trusted (§5.8.1). We document this bug for reproducibility.

3.7.4 EVM-mechanics fixes

Constructing multi-leg synthetic transactions on copied state surfaced three EVM state-management subtleties, each fixed and unit-tested:

- **"revision id N cannot be reverted."** Bracketing a `core.ApplyTransaction` in an outer `Snapshot`/`RevertToSnapshot` is illegal because `ApplyTransaction`'s own `Finalise` clears the journal's valid revisions. We removed the outer snapshot/revert and obtain isolation purely from a fresh `parent.Copy()` per sandwich probe (each probe is independent by construction).
- **"Refund counter below zero (gas > refund : 0)."** A read-only `vm.Call` leg (the V3 router) inherits a stale refund counter. We wrap each EVM leg with `statedb.Prepare(rules, attacker, coinbase, &router, ActivePrecompiles, nil)` before and `statedb.Finalise(true)` after, mirroring the canonical per-tx lifecycle so the refund accounting is well-formed.
- **Wrong-router routing.** An early version hardcoded the attacker to a Pancake router regardless of the victim pool, mispricing fork pools. The K-safe direct `pair.swap` to the victim's *actual* pool removes the assumption.

3.7.5 Cost gate

net = grossBNB − gasBNB − flashFeeBNB. Gas is the measured 3-leg cost (frontrun + backrun; the victim's gas is the victim's, not the attacker's) at the swept gas prices; the attacker is assumed flash-funded so the only capital cost is the flash premium on the numeraire borrowed for the frontrun (`flashFeeBNB`, V2 in-kind ≈ 0, Aave-style a few bps). As with backrun, every assumption is generous to the attacker, so the reported net-positive sandwich count is an *upper bound*.

3.8 The in-block counterfactual: ex-post existence vs. realized capture

Everything to this point measures *ex-post existence*—that an opportunity was present on-chain. The decision-relevant quantity is *realizable capture*: how much an independent searcher could actually take. The gap between them is exactly what an integrated, latency-advantaged competitor removes. We measure that gap directly, on the same ground-truth substrate, with an **in-block counterfactual**: for each ex-post opportunity our detector

surfaces in a canonical block, we ask whether a *real* competitor already captured it *in that same block*. Captured ⇒ unavailable; uncaptured ⇒ "left on the table" (still an upper bound—see §6.1).

3.8.1 Detecting a landed sandwich in the canonical block

The detector mode `SIMENGINE_DRYRUN=realizability` rides on the *same single* `ApplyOnStateHooked` replay used everywhere else (no extra EVM execution, no mutation of canonical state). In one pass it (a) runs the unchanged ex-post sandwich-any evaluation (§3.7) to enumerate the block's ex-post net-positive opportunities, and (b) builds a per-transaction ledger of hub-asset balance deltas and parses every V2/V3 `Swap` log into legs `(pool, txIdx, direction, sender, beneficiary, from-EOA, amounts)`. It then declares a **landed sandwich** on a pool only when a *conjunction* of strong signals holds—the design is deliberately biased so that, under any doubt, an opportunity falls to *left-on-the-table* rather than *captured* (over-counting capture would understate the realizable surface, the dangerous direction):

1. **Bracket structure.** A front leg and a back leg on the *same pool*, in *different* transactions with the victim between them, in *opposite* directions. JIT liquidity (Mint/Burn around the swap) is excluded.

2. **Same actor.** The two legs share a discriminating actor: the *same signing EOA* sent both bracket transactions ($\text{from}_{\text{front}} = \text{from}_{\text{back}}$—the strongest real-bot signal, since searchers route through their own contract but pay from one EOA), or a shared `Swap` sender/beneficiary that is neither a known router/aggregator nor the block coinbase.

3. **Profit corroboration (the false-positive guard).** The bracket must be a genuine round trip: net-*flat* in the volatile token and net-*positive* in the hub asset on *that pool*, net of gas and any coinbase bribe, above a USD dust floor. Hub profit is read over the actor's address *cluster* (signing EOA plus the legs' contract sender/beneficiary, minus routers/coinbase), because a real bot pays gas from its EOA while the proceeds accrue to its contract. A coincidental router routing two unrelated swaps fails this gate.

Captured opportunities are attributed to a **builder** (block coinbase / a builder contract registry), a **repeated MEV sender** (clustered across blocks), or **unknown**. The mode emits a `realizability tally` (ex-post, captured, left-on-table, capture rate, attribution) and a `realizability dist` (BNB

distributions of captured vs left), plus the funnel counters of the next paragraph.

3.8.2 An integrity episode (recall failure), and why the funnel is reported

The first implementation of this detector reported `captureRate = 0.00`—*zero* captured across hundreds of blocks. We did **not** report it: a zero capture rate is implausible given the literature on integrated BSC sandwichers, and an implausible-good result is exactly what the ground-truth discipline (§3.7.3) exists to catch—this time in the *opposite* direction (a false *zero* rather than a false *fortune*). The cause was a recall bug: the same-actor gate required the shared `Swap`-log actor to equal a leg's signing EOA, but real bots' `Swap` actor is their *contract*, never the EOA, so every bracket was dropped before corroboration. The fix (the $\text{from}_{\text{front}} = \text{from}_{\text{back}}$ signal plus cluster-based hub-profit corroboration) was verified by a unit test of precisely the integrated-bot pattern (contract actor, single signing EOA, profit in the contract) and by an independent, pool-agnostic scan of 1,500+ canonical blocks. To make any future near-zero *auditable* rather than blind, the detector permanently emits the funnel `bracketCandidates` → `sameActorPass` → `corroboratePass`, with the corroboration-failure breakdown (`notFlat / hubNeg / belowDust`). A near-zero capture rate is only credible when this funnel shows *why* each candidate legitimately fails—which §5.9 reports.

3.9 The censorship-differential: the one point-identified extraction estimand

The realizability counterfactual (§3.8) measures whether an ex-post opportunity was captured *in the block*. It does not, by itself, isolate the value an independent could *reach*. The naive "predicted-vs-sealed" contrast for total builder MEV is **non-identified**: private order flow is a *selected* treatment (correlated with state via routing) and per-transaction value depends on the whole ordering, so SUTVA fails. The one estimand that survives this is the **censorship-differential**

$$D = \mathbb{E}\big[\, V_i(1) - V_i(0) \;\big|\; o_i \text{ public},\ o_i \perp \text{ private flow},\ \text{builder dropped } o_i \,\big], \tag{1}$$

the receipt-exact value a builder leaves on the table by *dropping* (rather than including or internalizing) public, private-flow-orthogonal opportunities—the slice an independent party can, in principle, structurally reach. Measuring D adds one data dimension the rest of the paper lacks: the **public**

mempool, which supplies the treatment assignment (a public opportunity was included vs dropped).

3.9.1 Construction (`SIMENGINE_DRYRUN=censorship`)

The detector runs *live*, ingesting the public mempool via a `SubscribeTransactions` goroutine into a rolling, sender/nonce-indexed ledger (first-seen timestamp per hash). Per sealed block it builds inclusion/replacement indices from the canonical txs, runs one hooked replay to obtain the **post-sealed-block state**, and applies a conjunction of gates to every public candidate—every one biased so that *ambiguity excludes* (over-stating D is the dangerous direction; $\hat{D}$ is a strict lower bound):

1. **Available-at-seal**—recovered, correct-nonce (`GetNonce==tx.nonce`), valid and funded on the sealing parent; not nonce-gapped.
2. **Dropped, not replaced**—its `(sender,nonce)` slot is not filled by a different hash in the block.
3. **Net-of-gas profitable, self-contained**—valued by re-executing the candidate **alone on the post-sealed-block state** (§3.9.2), reading the executor's own BNB-denominated hub-asset delta net of gas; must emit $\geq$2 directional Swap legs (a single one-way swap's positive hub delta is sale proceeds, not profit) and clear a USD dust floor.
4. **Orthogonal to private flow**—no other sealed tx touches the opportunity pool (the SUTVA gate).
5. **Settled-never-mined** (§3.9.3)—the decisive gate.

3.9.2 Valuation on the post-block state (integrity episode 3)

A first implementation valued the candidate as the profit of *sandwiching* it—the **inverse** of D (a sandwich-the-user surface that over-states D by construction); caught in adversarial review and rewritten to the candidate's *own* realized net profit. A second version valued that own profit on the *pre-seal parent* state, in isolation—which counts opportunities the realized block had already closed (including by the builder's own internalized arb). We re-value on the **post-sealed-block state**: a candidate counts only if it is still profitable *after the entire builder block has executed.* This is conservative (it may under-state D: an opportunity profitable mid-block but closed by block-end is excluded) and it subsumes builder self-internalization for free (the internalizing tx is in the post-block state).

3.9.3 The settle window (integrity episode 4)

The load-bearing definition is "dropped". Defining it as "absent from *this* block" conflates *censored* with merely *pending*. We add a **settle window** of K blocks (default 256): a candidate is credited to D only if, after K subsequent blocks, its hash is *still absent from the canonical chain* (checked against a rolling mined-hash index; sender-nonce-advanced candidates are discarded as superseded). A candidate mined within the window is *delayed inclusion*, not censorship, and becomes an `includedComparable` control (the $T = 1$ arm). The detector defers the credit decision, freezing V_i at flag time and finalizing K blocks later. This is the gate that makes $\hat{D}$ a defensible lower bound rather than a count of pending transactions.

These two episodes join the units-bug (§3.7.3) and the realizability recall-bug (§3.8): in total we found and fixed four implementation bugs in the detectors during development, documented here for reproducibility.

4 Models: Naive Baseline to EVM Oracle

4.1 Naive baseline (v1)

Three deep all-18-decimal Pancake V2 pairs (WBNB/USDT, WBNB/USDC, USDT/USDC), fee 0.25% ($\gamma = 9975/10000$). Exact integer single-hop output; sqrt-free gate $\gamma_{\text{num}}^2 R_{aO} R_{bO} > \gamma_{\text{den}}^2 R_{aI} R_{bI}$; closed-form optimal input via integer floor-sqrt (unit-tested to exact wei); net = gross − gas − bid − margin with 250k gas, 3 gwei, bid 0, margin 0. Detection by a router-agnostic post-block `Swap`/`Sync` log scan. Result: 0/5,100 **blocks,** 0 **would-be profit**. Limits removed later: only 2-pool cycles among 3 hardcoded pools; only $x \cdot y = k$; post-block; analytic valuation.

4.2 Cross-DEX V2 graph (v2)

A token multigraph with edges carrying pool/DEX/γ, marginal rate, and weight $-\ln(r(1-\text{fee}))$ (detection only; valuation in `big.Int`). `NegativeCycles` enumerates cycles $2..K$ from WBNB; isolated all-V2 cycles are sized by the exact linear-fractional hop composition $f(x) = ax/(b+cx)$, $x^* = (\sqrt{ab}-b)/c$ (the exact generalization of the 2-pool optimum). Adding three Biswap V2 pools (fee 0.20%) yields 6 V2 pools and cross-DEX cycles. Result: 0 **Stage-A candidates** /2,950 **blocks** (snapshot spreads 0.029%/0.078% $\ll$ 0.45% fee threshold). Still post-block.

4.3 Intra-block detector (v3, v3+V3)

The correct trigger: after each transaction emitting a watched-pool `Swap` (V2 `Swap`/`Sync` plus the V3 `Swap` topic `0xc42079f9...`), snapshot reserves there and run the negative-cycle finder. V2-only: 0 **candidates** (V2 pools quiet). Adding six Pancake V3 pools (read via `ReadSlot0`/ `V3SpotPrice`) surfaces 803 **Stage-A candidates** /3,000 **sampled blocks**, *detected but unvalued* (`CycleOptimum` $= (0, 0)$ for V3 cycles)—the Stage-A/Stage-B split that justifies the oracle.

4.4 EVM oracle (v4) and cost / PBS model

Stage B (§3.5) values every candidate exactly. net = gross − `CycleGasUnits` · gasPrice − bid, capital-free flash funding [18], bid 0 in the headline (most generous). We emit the gross-profit distribution, break-even gas price, and the $\{0, 0.1, 0.3, 1, 3\}$ gwei sweep (`strategy/distribution.go`). **Realizable vs. ex-post (central caveat):** the backtest measures ex-post would-be gross against the post-swap transient; under BSC PBS [7, 20] a positive figure is an *upper bound* on realizable profit, so the thin ex-post net-positive residual we report (15 of 803) is an upper bound on what an independent searcher could realize, not a capture claim—that above-gas tail is the slice builders take first.

4.5 Sandwich model (any-pool)

The sandwich model reuses the exact same intra-block trigger as v4—every watched-pool victim swap—but, instead of searching for a trailing cross-venue cycle, it constructs and EVM-values the three-leg attack of §3.7 on the victim's own pool. Two design choices make it a *long-tail* instrument rather than a hub one:

1. **Any-pool decode.** A victim is any transaction emitting a V2 `Swap` (`0xd78ad95f...`) or V3 `Swap` (`0xc42079f9...`) on *any* pool, not only the verified 12-pool watch set—so the model sees the meme-coin/WBNB long tail where sandwiching actually concentrates. Pool metadata (token0/token1, reserves, fee family) is read from the runtime state of whatever pool emitted the event.

2. **Numeraire gate (§3.7.3).** Only pools with a hub-asset side (WBNB or a major stable) are valued, in BNB; pure token/token pools are `skippedNoNumeraire`.

The sizer is the γ-parameterized V2 sandwich closed form (`strategy/sandwich.go`) used *only* to seed; the profit authority is the EVM construction. net = grossBNB − gasBNB − flashFeeBNB, with the same $\{0, 0.1, 0.3, 1, 3\}$ gwei sweep. The funnel counters (`victimsSeen`, `skippedUnfundable`, `skippedUnsupported`, `skippedNoNumeraire`, `belowThreshold`, `grossPositive`, `netPositive`, `totalNetWei`) and the BNB-denominated distribution accumulator are tallied per processed block exactly as for backrun, so the two strategies are reported on identical scaffolding.

5 Results

All figures are produced by the read-only backtest harness on the synced bsc-geth v1.7.3 node, with the receipt-validated SimEngine as the profit oracle. The **backrun** experiments (§5.2) run with `SIMENGINE_DRYRUN=intrablock`; the **sandwich** experiments (§5.7) run with `SIMENGINE_DRYRUN=sandwich-any`; both take `SIMENGINE_DRYRUN_TALLY=N` and `SIMENGINE_DRYRUN_GASPRICES="0,0.1,0.3,1,3"`. Everything is strictly read-only; nothing is submitted; node/systemd/datadir are untouched; the binaries are built only to `/tmp` (`geth-sim10` for backrun, `geth-sim15` for the numeraire-corrected sandwich model). Backrun numbers are from the consolidated geth-sim10 run (3,000 sampled blocks; distribution from the last `intrablock dist` line, §5.6); sandwich numbers are from the consolidated geth-sim15 run (2,550 sampled blocks; funnel and distribution from the last `sandwich-any tally` and `sandwich-any dist` lines).

5.1 SimEngine validation

5.2 Model-evolution funnel

v1: 0/5,100, 0 would-be profit. v2: 0 candidates /2,950 (spreads $\ll$ fee threshold—no standing cross-DEX arb). v3: 0 candidates (major V2 pools quiet, $\sim$10 Pancake-V2 WBNB/USDT swaps per 30 blocks). v3+V3: 803 candidates detected but unvalued (V3 closed form intractable). v4: over 3,000 sampled blocks, watchedSwaps = 1,554, Stage-A = 803, EVM gross-positive = 803, net-positive = 15 at 3 gwei (1.9%)—$\sim$0.27 gross-positive cross-venue cycles per sampled block, $\sim$1 ex-post net-positive per 200 sampled blocks (0.017192 BNB $\approx$ \$10.1 total). These are ex-post figures, not a realizable-capture claim (§6.1).

Sampling. The v4 detector samples a representative subset of heads (per-candidate EVM valuation is slower than block production). Sampling is

incidental, not selective: a uniform-in-time subsample of consecutive heads. Counters are per processed block; the gross-positive rate is comparable across windows. The net-positive *rate* is window-sensitive (0.2% of gross-positive at the first 600 sampled blocks vs. 1.9% at 3,000), so we headline the 3,000-block figure and flag the variance.

5.3 Why post-block evaluation is not a backrun test

The v1/v2 zeros are real but not valid backrun tests. Both evaluate *post-block* reserves, where cross-venue prices are already re-aligned by intra-block competitors; backrun MEV lives in the transient state immediately after a victim swap, which end-of-block evaluation misses. The correct method (v3 onward) hooks in after each watched-pool swap and values at the only state where a backrun could exist. (The realizability/race question is separate; §6.1.)

5.4 Why almost all are sub-gas: gross-profit distribution and gas-sensitivity

There are ∼0.27 EVM-confirmed gross-positive cycles per sampled block; the result is that the overwhelming majority are below the gas cost (at 3 gwei only 15 of 803, 1.9%, clear it). From the `intrablock dist` log line (`strategy/distribution.go`, $O(1)$ memory: gross-USD and break-even-gwei in fixed 64-bucket, 0.25-decade log histograms, so percentiles are bucket *low edges*—true value in $[\text{edge}, \text{edge} \cdot 10^{0.25})$—while the `*_max` figures are exact running maxima). The gas floor is `gasUnits` $\cdot$ gasPrice $\approx$ 280k $\times$ 3 gwei $\approx$ 0.00084 BNB $\approx$ \$0.50.

RQ3—analytic over-count. Stage-A candidates / net-positive is ≈ 54 (803/15 at 3 gwei). The decisive over-count is gross→net: of 803 cycles an analytic marginal-rate or gross-only study would count as opportunities, only 15 (1.9%) survive once exact gas is applied to the EVM-true gross—a ∼54-to-1 over-count; the 15 are ex-post would-be net-positive, not realizable capture (§6.1).

5.5 Worked example: the representative net-positive backrun

The single largest / most representative net-positive cycle in the sample, as the EVM oracle valued it:

This is the structure the aggregates predict: a cross-version (V2↔V3) WBNB/USDT cycle whose EVM-true gross ($0.93) sits in the Table 4 tail above the $0.50 line, with break-even (5.64 gwei) above the detector's 3 gwei but below the population max (11.1 gwei). It clears gas by ∼$0.43 *ex-post*— not a claim a searcher could land that tx-97 slot ahead of the latency-advantaged builders (§6.1).

5.6 Provenance of the distribution figures

The backrun distribution figures are read from the last `intrablock dist` line of the consolidated 3,000-block run (`SIMENGINE_DRYRUN=intrablock SIMENGINE_DRYRUN_GASPRICES="0,0.1,0.3,1,3"`, attached to the synced node, read-only). Field-to-table mapping: Table 4 `grossPosSamples`, `grossUSD_p50/p90/p99/max`; Table 5 `breakevenGwei_p50/p90/max`; Table 6 `gasSweep_netPos` (`g=<gwei>:<count>(<pct>%)`); Table 7 `byDexMix` (`<label>:<count>`) and `byCycleLen` (`<n>hop:<count>`), % against `grossPosSamples`= 803. Tables 2–3 and the synthesis are independent of this line.

5.7 Sandwich results (any-pool, EVM-constructed)

The sandwich model (§3.7, §4.5) runs on the same node, same read-only harness, same intra-block victim trigger, with `SIMENGINE_DRYRUN=sandwich-any` and the same gas sweep. All figures below are from a single consolidated run over 2,550 **sampled blocks**; magnitudes were sanity-checked against the published BSC sandwich-MEV figure before being trusted (the units-bug episode of §3.7.3).

5.7.1 Sandwich funnel

Every gross-positive sandwich is on a **V2-family long-tail pool** (`byDex = v2_any:1735`): the construction's K-safe direct `pair.swap` covers any $x \cdot y = k$ fork, and that is where the volume—and the sandwiching—lives. The deep hub pools, which dominated the backrun watch set, contribute essentially nothing here.

5.7.2 Gross-profit distribution, break-even gas, and gas sweep

Percentiles are log-bucket low edges (true value in $[\text{edge}, \text{edge} \cdot 10^{0.25})$); maxima are exact. Source: the `sandwich-any dist` log line.
The gas-sweep counts the population with breakevenGwei $> g$ (gross of flash fee and bid); the headline **net-positive** = 1,162 applies the *full* gate

including the flash premium at 3 gwei, the ∼35-count gap being that premium. The contrast with backrun is stark on every line: where backrun's break-even median was **0.0032 gwei** (≈1/940 of 3 gwei) and only 1.9% cleared gas, sandwiching's break-even median is **10 gwei**—*above* the detector price—and **69.0% (1,197 of 1,735) clear the 3-gwei gas gate** (the headline net-positive 1,162 = 67.0% applies the full gate including the flash premium).

5.7.3 Worked example: a representative net-positive sandwich

The single largest net-positive sandwich in the sample reached **∼0.569 BNB (∼$330) net**; the distribution's exact gross max was **$430.50**—a long-tail tens-to-hundreds-of-dollars regime that simply does not exist for backrun on the deep hub (whose gross max was $1.85).

5.8 The backrun-vs-sandwich contrast (headline)

One instrument, one victim stream, one EVM valuation, one cost model—the two atomic strategies side by side (3-gwei gate):
Normalizing per sampled block, sandwiching yields **∼90× more net-positive opportunities** (0.46 vs 0.005 per block) and **∼2,000× more total value** (35.08 vs 0.0172 BNB) than backrun. The comparison is *controlled*: identical valuation on identical blocks, which the cross-paper literature comparison is not. The atomic-MEV edge for an independent searcher is, by this measurement, a **long-tail sandwiching** phenomenon—not the deep-pool cross-DEX backrun the arbitrage literature studies.

5.8.1 External sanity check, and the ex-post-vs-realized gap

As a *coarse* external check that the instrument is in the right magnitude regime—not an artifact—we extrapolate the ground-truth ex-post sandwich aggregate to a chain-year. The run sampled ≈58% of blocks (1.29 processed/s against the chain's ≈2.22 blocks/s at the 0.45-s Fermi block time); 35.08 BNB over 2,550 sampled blocks scales to a few hundred $M/yr at full coverage. This is the same order as widely-cited BSC MEV figures, **but we are deliberately careful with the comparator**: the much-quoted "$289M" is a 2025 *cross-context MEV volume* number (≈51.6% of a $561.9M total), **not** net-of-gas BSC sandwich profit; public dashboards (e.g. EigenPhi's BSC view, where arbitrage dominates) put *realized* BSC sandwich profit substantially lower. So the honest reading is not "we reproduced the published figure"—it is twofold:

1. **Order-of-magnitude plausibility.** Our extrapolated ex-post *gross* sits in the same coarse band as reported BSC MEV *volume*, which rules out a gross instrument-scale error (the units-bug class of failure, §3.7.3).

2. **The gap is the finding, not noise.** Our number is an *ex-post upper bound* (and is further inflated because the aggregate independently sandwiches *each* victim on a fresh state copy, summing concurrent same-pool attacks that could not all land, §6.1); reported *realized* sandwich profit is smaller. That wedge—ex-post upper bound well above realized capture—is precisely the quantity this paper exists to measure, and is exactly what the in-block counterfactual of §3.8 turns from a qualitative caveat into a number.

We therefore do **not** claim the $289M figure as validation of magnitude; we use it only to bound the order of magnitude, and we let the realizability instrument (§3.8 and its results) do the quantitative work.

5.9 Realizability: the in-block counterfactual

The §3.8 detector (`SIMENGINE_DRYRUN=realizability`) was run over 2,100 **sampled blocks**, after the recall-bug fix and validation (§3.8). The result is stark and stable across the run (identical at 450, 750, and 2,100 blocks):
The funnel makes the zero *auditable* rather than blind:
So same-actor brackets *do* occur (79 over 2,100 blocks—the fixed detector finds them), but **none is a profitable cross-tx sandwich**: 74 are not flat round trips (unrelated swaps that happen to share an actor) and 5 are net-*negative* in the hub (failed or out-competed bots, not captures). An independent pool-agnostic scan of 1,500+ canonical blocks corroborates this: genuine flat-round-trip-with-victim-between sandwiches are **essentially absent in the measured window**, and the recurring same-actor brackets are router-shared senders (correctly excluded) or non-round-trips. What *does* dominate realized BSC MEV in-window is **atomic, single-transaction** arbitrage/backrun (on the order of ∼7% of blocks carry an atomic opposite-direction round trip), a different category from the cross-transaction sandwich our ex-post surface enumerates.

Reading this correctly (and not over-reading it). A capture rate of 0 with 100% of the surface "left on the table" must **not** be read as "100% available to an independent searcher." Two facts forbid that reading. (i) *Scope:* this measures *cross-transaction sandwich* capture; the realized

MEV here is atomic single-tx arb, which our backrun track already showed is independently marginal (§5.2, 15/803 net-positive, ∼$10). (ii) *Mechanism:* the reason the ex-post sandwich surface is *unrealized* is that cross-tx sandwiching has receded from realized BSC activity—consistent with the validator/builder sandwich-filtering and private-flow regime of §6.4—and that same mechanism that removed the competitor sandwichers would also stop an independent's sandwich bundle from landing. *Unrealized is not the same as available.* The realizable independent sandwich edge is therefore ≈ 0, established now by *direct measurement* (no competitor is realizing the surface) **and** by mechanism (the surface is suppressed for everyone, us included)—a stronger, ground-truth statement than the ex-post upper bound alone.

Caveats (§6.1, §6.6). Single multi-hour window (a multi-day, multi-volatility window is future work); the detector scopes cross-tx sandwiches (atomic single-tx sandwiches are documented but not separately valued); corroboration conservatively requires the hub profit to still reside in the actor cluster at end-of-tx, so a bot sweeping proceeds to a cold address mid-transaction would be a (governing-rule-consistent) false negative; ∼1 nil-deref per 2–3 min in the shared ex-post EVM path is contained by per-victim `defer/recover` and drops the victim to the conservative left-on-table side (it cannot manufacture a capture).

5.9.1 Detector recall validation (the capture-0 is a measured null, not a blind detector)

A reading of "0 of 735 captured" requires ruling out that the detector simply has low recall for real landed sandwiches. We measure its recall directly with an injection harness (`SIMENGINE_DRYRUN=recalltest`): per canonical block, we execute a *genuine* synthetic landed sandwich (real frontrun → real clean victim → real backrun, via the same swap/funding primitives, producing real `Swap` legs and hub-balance deltas), splice it into the block's legs, and run the *identical* `detectLandedSandwiches` over the augmented set; recall = injected sandwiches detected. We sweep the structural axes that govern real-world recall and report recall per cell (≥211 injected per cell, three windows, recall stable to ±1%):
So on the structures that dominate BSC sandwich MEV—same-actor brackets (including the integrated-bot contract-sender pattern), cross-EOA shared-beneficiary, routed bots, and stable-hub brackets—the detector recovers **95–96%** of injected landed sandwiches at a **0% false-positive rate** on clean victims. Recall collapses to 0% only on the two documented blind spots

(profit swept to a cold address mid-transaction; round trips kept deliberately >2% non-flat) and to ∼52% on thin pools where realized profit is sub-dust (correctly floored by the $1 dust gate). The realized-capture count of 0/735 is therefore a *measured null at high recall* on the dominant structures, not an artifact of a low-recall detector; the residual false-negative surface is bounded to the two named blind spots, which we flag in §6.6.

5.10 Censorship-differential: the public residual is empty

The §3.9 detector (`SIMENGINE_DRYRUN=censorship`, settle window $K = 256$) was run live at the chain tip. After the four-gate conjunction and the chain-verified settle window, the result is decisive: $\hat{D} = 0$ **BNB, 0 settled drops**—no public, profitable, private-flow-orthogonal opportunity survived the builder's block *and* went unmined for K blocks. The live funnel (≈850 blocks) reads `publicOppsSeen`≈17, `droppedFromN`≈8, `csSkipMinedLater`/`skipSuperseded` absorbing the rest, `pendingDrops`≈0, `Dhat_count=0`; the few flagged candidates all finalized as *superseded* (sender nonce advanced—repriced, not censored).

The instrument's own diagnostic is the finding. We took the **958 unique tx hashes the pre-settle-window detector had logged as "drops"** (the population behind a spurious ∼47 BNB figure) and queried each against the canonical chain: **954 / 958 (99.6%) were actually mined later**—1 to 123 blocks afterward (≈1–90 s)—i.e. pure *delayed inclusion*; the remaining 4 (0.4%) have no chain record at all (most consistent with repricing, which the live superseded gate excludes). On post-PBS BSC, public arbitrage transactions that miss a block are **not censored—they are delayed** (mined a few blocks later) or repriced. The structurally-reachable public surface for an independent searcher is, by direct chain-verified measurement, **empty**.

Caveat (honest): the *live* candidate stream is thin (single-digit `droppedFromN` per hundreds of blocks), because genuine public, profitable, round-trip, orthogonal opportunities at the BSC tip are vanishingly rare post-PBS—itself consistent with the rest of the paper. The robustness comes from the large historical cross-check (958 hashes at 99.6% mined-later), not the thin live tail. The conservative direction is preserved throughout (every ambiguity discards). This is the causal-inference form of *unrealized* ≠ *available*: even the one point-identified, structurally-reachable estimand is zero.

5.11 Synthesis

Statistical precision (so no headline is a bare point estimate). Rates carry Wilson 95% intervals and the zero-event counts carry rule-of-three 95% upper bounds: backrun net-positive $15/803 = 1.87\%$ $[1.14, 3.06]$ of gross-positive cycles (per-block 0.50% $[0.30, 0.82]$); sandwich net-positive $1{,}162/1{,}735 = 67.0\%$ $[64.7, 69.1]$ (gas-only sweep 69.0% $[66.8, 71.1]$); censorship apparent-drops that were merely delayed-inclusion $954/958 = 99.6\%$ $[98.9, 99.8]$. For the two decisive zeros: realizability captured $0/735$ ex-post opportunities $\Rightarrow \leq 0.41\%$ captured (and 0 over 2,100 blocks $\Rightarrow \leq 0.14\%$ per block), which combined with the recall of 95–96% (§5.9.1) bounds the *realizable* cross-tx sandwich-capture rate well under 1%; and censorship genuine-censored $0/958 \Rightarrow \leq 0.31\%$. These are single-/multi-hour-window bounds; disjoint multi-day replication is a camera-ready strengthening (§6.7).

Three measurement angles, one instrument, one convergent verdict. **Backrun**: cross-venue gross-positive cycles on BSC's major V2/V3 pools are *frequent* (∼0.27 EVM-confirmed per sampled block, 803 over 3,000) and *real* (every nominated candidate confirmed gross-positive by the EVM), but only **15 (1.9%)** clear the ∼\$0.50 gas floor—∼1 per 200 sampled blocks, 0.0172 BNB (∼\$10) total; the gross→net collapse (803 → 15, ∼54-to-1) is the quantitative case for ground-truth valuation over analytic CFMM modeling, especially for V3. **Sandwich**: on the long tail the same instrument finds a *substantial* surface—**1,162 of 1,735 net-positive (67.0%) after gas+flash** (69.0% clear the gas-only 3-gwei gate) over 2,550 blocks, **35.08 BNB (∼\$20,200)**, median gross \$1.78, max \$430.50—∼90× the backrun net-positive rate and ∼2,000× its value, in the same coarse order as reported BSC MEV volume (§5.8.1, with the comparator caveat). Both results are *ex-post existence*, not realizable capture: the above-gas tail of either strategy is the slice latency-advantaged integrated builders (48Club, BlockRazor, the great majority of MEV) take first [20]. Both are credible because the profit is the EVM's own computation (5/5 receipt-exact validated), at the correct intra-block transient (§5.3), with V3 and any-fork pools valued by the actual pool bytecode rather than an over-counting $x \cdot y = k$ approximation. **Realizability** then supplies the third movement: the in-block counterfactual (§5.9) finds that **0 of the 735 ex-post sandwich opportunities in a 2,100-block window were actually captured by a real cross-tx sandwich**—the entire ex-post surface is *unrealized* in canonical blocks, because cross-tx sandwiching has receded from realized BSC activity (validator/builder filtering + private flow, §6.4) while realized MEV is atomic-arb-dominated. **Censorship-differential** supplies the third and final angle (§5.10): the one

point-identified, structurally-reachable estimand—the value a builder leaves by dropping public, orthogonal, profitable opportunities—is, after a chain-verified settle window, $\hat{D} = 0$; 99.6% of apparent public "drops" were merely *delayed inclusion* (mined a few blocks later), not censorship. These three angles converge on the verdict stated in full in the Abstract and §1.5: ex-post the atomic-MEV surface has *migrated from deep-pool backrun to long-tail sandwiching*, but *realized* none of it is capturable by an independent (backrun sub-gas, the sandwich surface captured by no one and filtered for us too, the public residual empty—*unrealized* $\neq$ *available*, §5.9, §5.10, §6.1), so the independent atomic-searcher edge is ≈ 0 measured from three angles. One ground-truth instrument supplies all three on the same footing, and across its four sub-studies we found and fixed four implementation bugs (§3.7.3, §3.8, §3.9.2, §3.9.3), documented for reproducibility.

6 Discussion, Limitations, Threats, and Future Work

The central finding restated: measured on one ground-truth instrument and one victim stream, post-PBS (2026), the two atomic MEV categories diverge. On BSC's major V2/V3 WBNB/stablecoin pools, cross-venue **backrun** arbitrage *exists ex-post and is frequent* ($\sim$0.27 EVM-confirmed gross-positive cycles per sampled block, 803 over 3,000) but is *overwhelmingly sub-gas*—only 15 of 803 (1.9%) clear the $\sim$\$0.50 / 280k-gas $\times$ 3-gwei floor. **Sandwiching the long tail** is the opposite: 1,162 of 1,735 ex-post net-positive over 2,550 blocks (35.08 BNB $\approx$ \$20,200), $\sim$90$\times$ the backrun net-positive rate and $\sim$2,000$\times$ its value. The independent-searcher atomic-MEV edge has thus *migrated* from deep-pool backrun (gone) to long-tail sandwiching (substantial). Both figures are *ex-post existence*, not realizable capture—the above-gas tail of either strategy is what latency-advantaged builders take first.

6.1 Realizable vs. ex-post

The backtest measures **ex-post existence**, uniformly for both strategies: given the realized ordering, a backrun inserted in the transient slot after a victim swap—or a sandwich wrapped around it—*would have had* gross p, as the EVM computes it. We **do not** prove a searcher could *win the race* to that slot. The gap is asymmetric in our favour: our ex-post figure is an **upper bound** on realizable profit (a searcher who does not control ordering, competes in $\sim$100–400 ms, and sees public flow after builders [20, 7] can capture *at most* the ex-post figure). So both the thin backrun residual (15

of 803 cycles, ∼$10) and the substantial sandwich surface (1,162 of 1,735, ∼$20,200) are *upper bounds* on what an independent searcher could realize, not demonstrations of capture—the above-gas tail of either is precisely the slice the latency-advantaged integrated builders (48Club, BlockRazor, ∼90% of MEV) capture first. This caveat is *more* binding for sandwiching, not less: sandwiching is the builders' single most lucrative atomic strategy, so the long-tail surface we measure is exactly what they (and their integrated searchers) contest most aggressively. The positive-existence side is reported as *would-be / oracle* net with no capture claim. The one structurally plausible path for an independent party to realize any of it is to **submit through the PBS builder API / order-flow auctions**—the disarmed Phase-3 path documented but never exercised.

From argued upper bound to measured gap. §3.8/§5.9 turn this caveat from an argument into a measurement. The in-block counterfactual finds the realized capture of our ex-post *sandwich* surface to be **zero** over 2,100 blocks (0 of 735), with an auditable funnel showing the absence is real (1,077 brackets → 79 same-actor → 0 profitable round-trips) and an independent 1,500-block scan confirming cross-tx sandwiches are essentially absent in-window. This sharpens the realizability statement in a way worth stating precisely, because it cuts *against* a naïve optimistic reading: a 0% capture rate does **not** mean the surface is available to us. The surface is *unrealized* because cross-tx sandwiching has receded from realized BSC activity (filtering + private flow, §6.4); the very mechanism that removed the competitor sandwichers—bundles of that shape are excluded at the production layer, and victim flow is increasingly private—is what would stop *our* sandwich from landing too. **Unrealized is not available.** So the realizable independent sandwich edge is ≈ 0 by *two* independent arguments now: the ex-post-upper-bound logic above, and the direct observation that no party is realizing the surface under a regime that would exclude us identically. For *backrun* the realizability gap is the latency/ordering race already discussed (and the surface is sub-gas regardless); for *sandwich* it is suppression at the production layer. Either way the measured realizable edge for an independent is indistinguishable from zero.

A second, sandwich-specific upper-bound source: the aggregate sandwiches *each* victim independently on a fresh state copy, so two sandwiches targeting victims in the same pool and block are both counted at full value even though, executed for real, the first would move the pool and shrink the second. The per-sandwich figures are exact; the *aggregate* (35.08 BNB)

therefore over-counts concurrent same-pool opportunities and is best read as an upper bound on the total surface, consistent with its landing ≈2× above the published realized figure (§5.8.1). Backrun is largely free of this effect (cross-venue cycles on distinct deep pools rarely collide), which makes the ∼2,000× value gap, if anything, an *under*-statement of how concentrated the realized edge is in sandwiching.

6.2 Watch-set scope (and why the two strategies are scoped differently)

The two measurements deliberately have *different* scope, matching where each strategy's surface lives. **Backrun** runs on a verified 12-pool set—three deep Pancake V2 hub pairs, three Biswap V2 mirrors, six Pancake V3 pools (tiers 100/500/2500) across WBNB/USDT, WBNB/USDC, USDT/USDC—the economically dominant hub [20], where an independent backrun edge, if it existed on liquid markets, would be most defensible to claim; the result there is the near-null (15 of 803). **Sandwich** is *not* confined to that set: the any-pool decoder (§4.5) attacks any V2-fork pool a victim touches, so it covers the **long tail** of thin/volatile/new WBNB-paired pools—and that is precisely where its entire surface turned out to be (`v2_any:1735`, zero on the deep hub), confirming the hint from the backrun runs that the deep pools are quiet (e.g. V3 WBNB/USDT fee-100 ≈ 0 swaps per 30 blocks) while the action is in the tail. This is the study's structural punchline: the long tail is not a "future work" gap but the *locus of the surviving edge*, and the sandwich instrument measures it directly.

What remains genuinely out of scope: non-Pancake V3 / Algebra pools (counted `skippedUnsupported`, ∼35% of victims—a routing gap, not a valuation one), pure token/token pools with no BNB numeraire (`skippedNoNumeraire`), tokens with non-standard `balanceOf` layouts (`skippedUnfundable`), other pool types (Thena CL, stableswap, V4 hooks), and multi-block / cross-domain / CEX-DEX strategies (out of scope for an *atomic* study). Every exclusion is *conservative*—it can only lower the measured sandwich count—so the surface is, if anything, larger than reported. The scoped claims are therefore: *no realizable net-positive independent backrun on the major V2/V3 hub pools* (only a thin ex-post residual of 15 of 803), and *a substantial ex-post sandwich surface on the long tail* (1,162 net-positive, an upper bound on realizable capture)—together, *the independent atomic-MEV edge is a long-tail sandwiching phenomenon*, not a statement that no MEV exists anywhere on BSC.

6.3 Sampling

The sampling is incidental and uniform-in-time, uncorrelated with opportunity. Counters are per processed block; the gross-positive rate is comparable across windows. The net-positive *rate* is window-sensitive (0.2% of gross-positive at the first 600 sampled blocks vs. 1.9% at 3,000), because the tail is thin and a single large cycle moves a short-window percentage, so we headline the 3,000-block figure and report the variance. We claim a rate, a distribution, and a thin ex-post net tail, not a chain-wide count.

6.4 The builder / PBS capture explanation

We interpret the result through the post-BEP-322 regime [7, 20] (two builders, ∼90% of MEV): the largely sub-gas gross distribution is what an efficient market looks like after integrated searcher-builders have arbitraged the deep pools to the fee/gas boundary within each block; most of what remains at the transient is the residual not worth their near-zero marginal cost—exactly the slice that cannot cover an external searcher's full gas. The thin above-gas tail that does exist ex-post (the 15 net-positive cycles) is, on this reading, precisely what the latency-advantaged builders take first—an external searcher, seeing public flow tens of ms later and not controlling ordering, cannot reliably win that slot. This is interpretation, not direct measurement; the instrument measures the gross/net distribution, and the builder attribution is inference.

The sandwich result fits the same regime from the other side. The long-tail sandwich surface is *large* ex-post precisely because the long tail is where the deep-pool efficiency argument does *not* apply: thin memecoin pools have wide price impact, new listings, and unsophisticated victims, so a manufactured spread is both larger and more frequent than on the hub. That this surface exists ex-post is therefore expected; that an *independent* searcher could capture it is not—sandwiching is the single most contested atomic strategy for the integrated builders, who see the victim's swap in private flow first and can place the frontrun with certainty of ordering. So the ∼$20,200 sandwich surface and the ∼$10 backrun surface tell one consistent story: the deep hub is arbitraged flat, the long tail carries the value, and builder integration captures the above-gas tail of both—leaving the independent searcher an ex-post upper bound far above any realizable figure.

Two concurrent 2025–2026 developments make this interpretation concrete and, if anything, *tighten* the realizability gap for sandwiching specifically. First, the dominant BSC builders now run **in-house arbitrage**

at industrial scale: the builder-ecosystem census [20] attributes tens of thousands of arbitrage contracts to the two leading builders and millions of builder-executed cycles, i.e. the builders self-execute the atomic MEV rather than merely auctioning blockspace—so an external bundle is racing the block's own producer, which sees private flow first and decides its cut after merging. Second, a validator/builder *Goodwill Alliance* has begun **filtering sandwich-pattern bundles** at the production layer: GWA-aligned builders proactively exclude buy→buy→sell sandwich bundles and active validators enforce this, with operator-reported reductions in daily sandwich activity of roughly two orders of magnitude [8] (we cite the magnitude as self-reported, not independently verified, but the *direction*—enforced sandwich filtering across most blocks—is well attested). The implication for our headline is sharp: the long-tail sandwich surface we measure is *real ex-post*, but it is exactly the bundle shape an increasing fraction of block production now rejects, and the victim flow is increasingly routed through MEV-protect/private RPCs that never reach the public mempool an independent searcher observes [6]. Our ex-post figure is thus an upper bound on a surface that is *additionally* being closed off at the protocol-operator layer—which is why we treat the realizable fraction (§3.8) as the decisive quantity, not the ex-post aggregate.

6.5 Gas, bid, and cost modeling

For backrun, net = gross − $\sum$gas − bid, gas = measured per-cycle units (∼280k) at 3 gwei, bid = 0 in the headline (most generous). For sandwich, net = grossBNB − gasBNB − flashFeeBNB (§3.7), the same gas sweep, flash premium charged on the borrowed numeraire. Every assumption in both is *generous to the searcher*: a deployed executor adds fixed overhead (only raises the floor); the gas-price *sweeps* (Tables 6, 12) show neither conclusion is knife-edge on one price; any positive bid only lowers net; capital is free via flash funding (V2 in-kind premium 0; Aave premium only raises the floor); and the any-fork sandwich under-quotes the attacker's output by computing it at the Pancake 0.25% fee (§3.7), which can only *lower* sandwich gross. Because every assumption is generous to the searcher, both the 15 ex-post net-positive backrun cycles *and* the 1,162 net-positive sandwiches are *upper bounds*: tightening any assumption can only shrink the counts, not grow them. For backrun this makes the sub-marginality conclusion robust; for sandwich it means the true realizable figure sits *below* a surface that is already ∼2× the published realized aggregate (§5.8.1), consistent with heavy builder capture.

6.6 Threats to validity (consolidated)

- **(T1) Ex-post vs. realizable**—§6.1: ex-post is an upper bound for *both* strategies; the 15 backrun cycles and the 1,162 sandwiches are would-be, not realized, and the above-gas tail of either is builder-captured first (more binding for sandwiching, the builders' most contested atomic strategy).

- **(T2) Scope**—§6.2: backrun scoped to the verified 12-pool hub; sandwich covers the long tail (any V2-fork pool) but excludes non-Pancake V3/Algebra (`skippedUnsupported`), token/token pools (`skippedNoNumeraire`), and odd-slot tokens (`skippedUnfundable`)—all conservative (undercount) exclusions.

- **(T3) Evaluation point**—the headline experiments evaluate at the post-swap intra-block transient (correct backrun point); the post-block v1/v2 zeros are flagged as standing-arb measurements (§5.3).

- **(T4) Cost-model assumptions**—§6.5: all generous; gas swept; bid 0; capital-free.

- **(T5) Sampling**—§6.3: uniform-in-time subsample; gross-positive rate stable; net-positive rate window-sensitive (0.2% $\rightarrow$ 1.9%), so the 3,000-block figure is the headline.

- **(T6) Validation sample size**—5/5 is a strong certificate (every field on 100+ txs per block) but small; the protocol scales trivially; the oracle's V2 closed form agreeing with the chain's swap math is also receipt-validated.

- **(T7) Stage-A recall**—false positives eliminated by EVM-valuing every candidate; the residual risk is false negatives (size-profitable-but-marginally-unprofitable cycles); given all but 15 EVM-valued candidates are sub-gas and marginal profitability is necessary for the small hub cycles, the risk is low for the watched set but is a genuine limitation for the general claim (motivates a recall study, §6.7).

- **(T8) Oracle fidelity for V3**—V3 backrun gross is QuoterV2 on the exact intermediate state (gross is ground truth; gas is the per-hop estimate). A fully synthetic-tx executor (`verifyCycleEVM`, stubbed) would only *raise* the gas side and so could only *shrink* the 15-cycle ex-post residual, not enlarge it.

- **(T9) Sandwich aggregate over-count**—§6.1: sandwiches are constructed independently per victim on fresh state copies, so concurrent same-pool/same-block opportunities are summed at full value; the *per-sandwich* figures are exact, but the 35.08-BNB *aggregate* is an upper bound (consistent with its ≈2× overshoot of the published realized figure, §5.8.1). Per-block *rates* and the distribution are unaffected.

- **(T10) Any-fork fee assumption**—§3.7: the K-safe direct `pair.swap` computes the attacker's output at the Pancake 0.25% fee even on lower-fee forks, *under*-quoting attacker gross—a conservative bias that can only lower the sandwich count.

- **(T11) Sandwich routing coverage**—non-Pancake V3 / Algebra victims (∼35%, `skippedUnsupported`) are not yet routed, so the sandwich surface is *under*-counted; closing this gap (§6.7) can only enlarge it, never shrink the contrast with backrun.

- **(T12) Token funding fidelity**—sandwich funding writes ERC20 balance/allowance slots directly; tokens whose layout is not reproduced are `skippedUnfundable` (under-count). For funded tokens the EVM then applies the real transfer logic (including fee-on-transfer), so funded-token valuation is ground truth.

- **(T13) Realizability detector recall**—the capture-0 result is only meaningful if the detector would catch real landed sandwiches. The injection harness of §5.9.1 measures **95–96% recall** on the dominant same-actor structures (incl. the integrated-bot contract-sender pattern, cross-EOA, routed, and stable-hub brackets) at a **0% false-positive rate**, with recall collapsing only on two documented blind spots (mid-tx proceed-sweeping to a cold address; deliberately >2%-non-flat round trips) and ∼52% on sub-dust thin pools. So 0/735 is a measured null at high recall on the structures that dominate BSC sandwich MEV, with a bounded false-negative surface.

6.7 Future work

The realizability result reprioritizes the roadmap. (1) **Longer, multi-volatility realizability windows.** Our capture-rate-0 result is a single multi-hour window; a multi-day window spanning listings/depegs would confirm whether realized cross-tx sandwiching stays ≈ 0 or spikes in stress regimes (and would test the GWA-suppression hypothesis of §6.4 over time).

(2) **Atomic single-tx sandwich/arb valuation.** The realized-MEV scan shows atomic single-tx activity dominates; valuing it on the same instrument (it does not fit the cross-tx victim-bracket model) would close the one realized category we currently only count, not value. (3) **Tighten the counterfactual against the mid-tx-sweep false negative** (a bot moving proceeds to a cold address before tx end), e.g. via intra-tx trace-level balance tracking, to confirm the 0 is not hiding swept captures. (4) **Close the sandwich routing gap** (non-Pancake V3 / Algebra, ∼35% `skippedUnsupported`)—pure upside to the *ex-post* surface. (5) **Extend backrun to the long tail** with automated pool discovery, to test whether *backrun* too has a long-tail ex-post surface the 12-pool hub set misses. (6) Complete **end-to-end synthetic-tx verification** for backrun (`verifyCycleEVM`). (7) Model (and, only in a sanctioned setting, exercise) the **PBS-builder submission path**—the only avenue by which any ex-post surface could in principle become capturable, and the cleanest live falsification test of the realized-≈0 conclusion.

6.8 What would change the conclusion

The conclusion has three falsifiable layers. The **ex-post backrun** finding (deep-pool surface sub-gas) would be overturned by: (1) a materially larger above-gas backrun population at a realistic BSC gas price; or (2) a flaw in the oracle under-stating backrun gross. The **ex-post sandwich** finding (large long-tail surface) would be overturned by a routing/funding bias that *over*-states sandwich gross (we argue every such bias is conservative, §3.7, T10–T12). The **realizability** finding—the decisive one, that realized independent capture is ≈ 0—would be overturned by: (3) a longer or different window in which the in-block counterfactual finds *non-trivial* cross-tx sandwich capture (our 0/735 is one multi-hour window); (4) evidence that the counterfactual's conservative corroboration is hiding real captures (e.g. mid-tx proceed-sweeping, T9/§6.7), which intra-tx trace tracking would expose; or (5) a demonstrated independent path that actually *lands* value against the suppression—the PBS-builder submission route exercised in a sanctioned test (§6.7) returning net-positive realized profit. The **comparative ex-post** claim (sandwiching $\gg$ backrun) would be overturned only if a backrun long-tail turned out comparably large, which the volume asymmetry makes unlikely. Note the layers interact: even a large ex-post sandwich surface (which we *do* find) does not resurrect the edge, because realizability is measured separately and is ≈ 0. None of the overturning conditions is observed; all are concrete and testable, which is what makes "the independent atomic edge is ≈ 0" a falsifiable claim.

6.9 Model capacity is not the bottleneck (an ML robustness check)

A natural objection: maybe a *bigger* model—a deep net on an H200 rather than gradient-boosted trees—would find a predictive edge our measurement missed. It cannot, for a structural reason, and a capacity ladder confirms it empirically.

The oracle bound (structural, capacity-independent). Our receipt-exact counterfactual is not a model to be scaled—it *is* the oracle predictor, granting perfect ex-post information, the strongest predictor that can exist. Its realizable atomic value is ≈ 0. Hence for any predictor f on any input representation, value(f) $\leq$ value(oracle) ≈ 0: the binding constraint is *acting* (ordering, latency, private flow, atomicity, gas), not *predicting*, and no parameter count changes the structure. A predictive model cannot exceed perfect information.

A capacity ladder (empirical). On the two structurally *learnable* proxy targets—T1 curl-magnitude (`curlFrac`, non-commuting subset, $n = 359$) and T2 value-magnitude ($\log V$, $n = 7{,}680$)—we ran a model-capacity ladder under identical GroupKFold-by-block CV (all preprocessing fit in-fold; out-of-fold R^2 only; near-label/post-treatment leaks excluded):
CV performance is **flat / non-increasing in capacity**. On T1 the linear anchor already recovers essentially everything and the tree optimum is the *shallowest* depth in the grid; on T2 **XGBoost is the best model and no deep net beats it**—the regime where gradient-boosted trees match or exceed deep tabular nets [14]. The only lever that moved T2 was *representation* (pool one-hot/frequency-encoding: $0.09 \rightarrow 0.65$), not capacity. XGBoost was not "too small".

Degenerate extraction targets. The targets that would matter for *extraction* carry no positive mass: realized "captured" has 0 positives (every model, from a constant to an H200-scale net, fits it with zero error and undefined AUC); "real-censored" has 7/7,680 positives, and a sweep from 577 to $\sim$800k parameters *worsens* AUC ($0.965 \rightarrow 0.758$, i.e. bigger overfits), with the seven "positives" being delayed-inclusion artifacts (AUPR ≈ 0.22 at AUC ≈ 0.97), not positive-EV opportunities. Capacity is irrelevant where there is nothing to separate.

Robustness corollary (ML used legitimately). Used *within* the identified estimand (Invariant E: learned models as nuisances, never as reported scalars), XGBoost as the AIPW outcome/propensity model leaves the censorship-differential indistinguishable from zero: the AIPW point is non-identified (positivity fails—overlap collapses), and the defensible exact-V differential is $D = -0.029$ BNB, bootstrap 95% CI $[-0.178, +0.081] \approx 0$, corroborating $\hat{D} = 0$. RL is excluded a priori: with realizable reward $\equiv 0$ there is nothing to optimize. **The H200 is the right instrument only for a different problem** (non-atomic statistical arbitrage on a contestable venue, where prediction creates actionable, non-oracle-bounded edge)—not for the BSC atomic question, whose realizable edge is structurally pinned at zero.

Reproducibility statement

All addresses, storage slots (including the ERC20 balance/allowance slots and the runtime slot-probing used for sandwich funding), event signatures, fee tiers, and pool reserves used in this study were verified directly against the project's own synced bsc-geth v1.7.3 node (chainId = 56, head ∼block 102.54M)—the same node that produces the ground-truth execution—so the watch set and all measurements are reproducible from the node alone, with no third-party API dependency. The instrumented binaries are built only to `/tmp` (`geth-sim10` for backrun, `geth-sim15` for the sandwich model, `geth-sim16` for the realizability counterfactual, `geth-sim17` for the censorship-differential); the experiments are strictly read-only and never submit; the running node, its systemd unit, and its datadir are untouched. The backrun figures are from a 3,000-sampled-block `intrablock` run; the sandwich figures from a 2,550-sampled-block `sandwich-any` run; the realizability figures from a 2,100-sampled-block `realizability` run (with an auditable bracket→same-actor→corroboration funnel and an independent 1,500-block pool-agnostic cross-check); the censorship-differential from a live `censorship` run with a $K = 256$-block settle window, cross-checked by querying the canonical chain for the later inclusion of every flagged drop (954/958 historical drops verified mined-later).

References

[1] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. Technical report, Uniswap Labs, March 2021.

[2] Guillermo Angeris, Akshay Agrawal, Alex Evans, Tarun Chitra, and Stephen Boyd. Constant function market makers: Multi-asset trades via convex optimization. *arXiv preprint arXiv:2107.12484*, 2021.

[3] Guillermo Angeris and Tarun Chitra. Improved price oracles: Constant function market makers. In *2nd ACM Conference on Advances in Financial Technologies (AFT)*, 2020. arXiv:2003.10001.

[4] Guillermo Angeris, Tarun Chitra, Alex Evans, and Stephen Boyd. Optimal routing for constant function market makers. In *Proceedings of the 23rd ACM Conference on Economics and Computation (EC '22)*, 2022. arXiv:2204.05238.

[5] Kushal Babel, Mojan Javaheripi, Yan Ji, Mahimna Kelkar, Farinaz Koushanfar, and Ari Juels. Lanturn: Measuring economic security of smart contracts through adaptive learning. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*, pages 1212–1226, Copenhagen, Denmark, 2023. ACM. ePrint 2023/1338; black-box adaptive-learning optimizer whose output transactions execute on-chain without modification on a simulation framework that runs over real blockchain state and contract bytecode; evaluated on Sushiswap/UniswapV2/V3/AaveV2 for sandwich, arbitrage, backrun. Nearest executing (non-analytic) prior art; per-target rather than per-block-census.

[6] BlockSec. How is the performance of BSC after full implementation of PBS? `https://blocksecteam.medium.com/how-is-the-performance-of-bsc-after-full-implementation-of-pbs-f720114d6fdd`, 2025. Reports post-PBS builder concentration (duopoly) and trend toward monopoly.

[7] BNB Chain. BEP-322: Builder API specification for BNB smart chain. bnb-chain/BEPs Pull Request #322, 2024.

[8] BNB Chain. How the goodwill alliance slashed sandwich attacks by 95%. `https://www.bnbchain.org/en/blog/how-the-goodwill-alliance-slashed-sandwich-attacks-by-95`, 2025. BNB Chain self-reported; GWA-aligned builders filter sandwich-pattern bundles, validators enforce. Figures (140K to <1K daily attacks) are operator-reported.

[9] Tianyang Chi, Ningyu He, Xiaohui Hu, and Haoyu Wang. Remeasuring the arbitrage and sandwich attacks of maximal extractable value

in ethereum. *arXiv preprint arXiv:2405.17944*, 2024. Post-Merge Ethereum measurement; profitability-identification algorithm and robust detection for arbitrage and sandwich MEV (6.33M arbitrages, 3.02M sandwiches). Log/heuristic detection, not re-execution – a measurement/taxonomy comparator for §2.

[10] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges. In *2020 IEEE Symposium on Security and Privacy (S&P)*, 2020. arXiv:1904.05234.

[11] Theo Diamandis, Guillermo Angeris, and Alan Edelman. Convex network flows. *arXiv preprint arXiv:2404.00765*, 2024.

[12] Theo Diamandis, Max Resnick, Tarun Chitra, and Guillermo Angeris. An efficient algorithm for optimal routing through constant function market makers. In *Financial Cryptography and Data Security (FC)*, 2023.

[13] Krzysztof Gogol, Manvir Schneider, Jan Gorzny, and Claudio Tessone. How to serve your sandwich? MEV attacks in private L2 mempools. *arXiv preprint arXiv:2601.19570*, 2026. attacker-profitability model for CPMMs and concentrated-liquidity AMMs (Uniswap/Pancake v3); derives the optimal sandwich strategy.

[14] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022. arXiv:2207.08815.

[15] Lioba Heimbach and Roger Wattenhofer. SoK: Preventing transaction reordering manipulations in decentralized finance. In *4th ACM Conference on Advances in Financial Technologies (AFT)*, 2022. arXiv:2203.11520.

[16] Robert McLaughlin, Christopher Kruegel, and Giovanni Vigna. A large scale study of the ethereum arbitrage ecosystem. In *32nd USENIX Security Symposium*, 2023.

[17] Kaihua Qin, Liyi Zhou, and Arthur Gervais. Quantifying blockchain extractable value: How dark is the forest? In *2022 IEEE Symposium on Security and Privacy (S&P)*, 2022. arXiv:2101.05511.

[18] Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. Attacking the DeFi ecosystem with flash loans for fun and profit. In *Financial Cryptography and Data Security (FC)*, 2021. arXiv:2003.03810 (2020).

[19] Christof Ferreira Torres, Ramiro Camino, and Radu State. Frontrunner jones and the raiders of the dark forest: An empirical study of frontrunning on the ethereum blockchain. In *30th USENIX Security Symposium*, 2021. arXiv:2102.03347.

[20] Qin Wang, Ruiqiang Li, Guangsheng Yu, Vincent Gramoli, and Shiping Chen. MEV in binance builder. *arXiv preprint arXiv:2602.15395*, 2026.

[21] Ye Wang, Yan Chen, Haotian Wu, Liyi Zhou, Shuiguang Deng, and Roger Wattenhofer. Cyclic arbitrage in decentralized exchanges. In *Companion Proceedings of the Web Conference 2022 (WWW '22)*, 2022. arXiv:2105.02784 (2021).

[22] Yu Zhang, Tao Yan, Jianhong Lin, Benjamin Kraner, and Claudio J. Tessone. An improved algorithm to identify more arbitrage opportunities on decentralized exchanges. *arXiv preprint arXiv:2406.16573*, 2024.

[23] Liyi Zhou, Kaihua Qin, Christof Ferreira Torres, Duc V. Le, and Arthur Gervais. High-frequency trading on decentralized on-chain exchanges. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 428–445, 2021. arXiv:2009.14021; first to formalize and quantify sandwich attacks.

Table 1: What we adopt vs. scope out from prior work.

Technique	Source	We use it as	Adopt / Scope-out
Negative-cycle (Bellman–Ford/SPFA) on $-\ln(r(1-\text{fee}))$	classical; [21]	candidate generator only	**Adopt**
Line-graph + Modified MBF	[22]	many-cycle enumeration	**Adopt (optional path)**
Two-pool closed-form optimal input	UniswapV2; `arb.go`	exact V2 sizer	**Adopt (implemented)**
Convex optimal routing	[4]	multi-path/cross-DEX sizing	**Adopt for coupled cycles**
Dual decomposition / per-CFMM oracle	[12]	implementable multi-pool engine	**Adopt for coupled/split**
Convex network flows (hypergraph)	[11]	most-general split routing	**Scope-out (future)**
V3 tick math	[1]	spot-price candidate detector only	**Adopt detector; defer profit to EVM**
Detect-then-EVM-value	this work; [16]	core methodology	**Adopt (our contribution)**
Flash swaps / flash loans	[18]	capital-free cost model	**Adopt (assumption)**
Sandwich formalization + CP optimal frontrun	[23]	seed sizer; victim-impact model	**Adopt; profit deferred to EVM**
V3/CLMM sandwich profitability	[13]	confirms no V3 closed form	**Adopt detector; defer profit to EVM**
BSC PBS reality	[7, 20]	regime framing for the null	**Adopt (interpretation)**
Adaptive-learning executing MEV optimizer	[5]	nearest non-analytic prior art (per-target executor)	**Contrast (per-block census vs. per-target optimizer)**
Heuristic arbitrage+sandwich re-measurement	[9]	recent large-scale measurement comparator	**Contrast (detection vs. re-execution)**

Table 2: SimEngine validation against stored receipts.

Metric	Value
Validation method	replay real mainnet block, compare to stored receipts
Fields compared	Status, GasUsed, CumulativeGasUsed, per-log Address/Topics/Data
Blocks validated	5 (around height ∼101.83M)
Transactions per block	100–151
PASS / FAIL	**5 / 0 (5/5 PASS)**
Mismatches	none

Table 3: Model-evolution funnel: naive to EVM oracle. All under the same validated SimEngine substrate.

#	Model	Scope	Eval. point	Range	Stage-A	Gross+	
v1	Naive 2-pool	3 Pancake V2	post-block	5,100	—	—	
v2	Graph cross-DEX V2	6 V2	post-block	2,950	**0**	0	
v3	Intra-block V2	6 V2	post-swap	(val. win.)	**0**	0	
v3+V3	Intra-block + V3 detect	12 pools	post-swap	3,000 samp.	**803**	n/a	
v4	Intra-block + EVM oracle	12 pools	post-swap	3,000 samp.	**803**	**803**	**15 @**

Table 4: Gross-profit distribution of gross-positive cycles. Percentiles are log-bucket low edges; max is exact.

Statistic	Gross (USD)	Relative to \$0.50
grossPosSamples (count)	803	—
grossUSD p50	\$0.00056	0.0011×
grossUSD p90	\$0.0056	0.011×
grossUSD p99	\$1.0	2.0×
grossUSD max (exact)	\$1.85	3.7×

Table 5: Break-even gas-price distribution. p50/p90 bucketed; max exact. A cycle is net-positive at price g iff breakevenGwei $> g$.

Statistic	Break-even (gwei)	vs. 3 gwei
breakevenGwei p50	0.0032	∼1/940 of 3
breakevenGwei p90	0.032	∼1/94 of 3
breakevenGwei max (exact)	11.1	3.7× the 3 gwei price

Table 6: Gas-price sensitivity sweep: count and % of gross-positive cycles that would be net-positive at each price (bid = margin = 0). At $g = 0$ the share is 100% by definition.

Gas price (gwei)	Would-be net+ (count)	Share (%)
0	803	100.0
0.1	63	7.8
0.3	45	5.6
1	29	3.6
3 (detector default)	15	1.9

Table 7: Structural breakdown of the gross-positive population by DEX mix (`byDexMix`) and cycle length (`byCycleLen`).

DEX-mix label	Count	Share (%)
biswap_v2+pancake_v3	327	40.7
pancake_v3×pancake_v3 (cross-tier)	254	31.6
pancake_v2+pancake_v3	222	27.6
Hops	**Count**	**Share (%)**
2	506	63.0
3	297	37.0
4	0	0.0

Table 8: Worked example of an ex-post net-positive backrun.

Field	Value
Block	102,461,418
Tx index	97
Victim tx	`0xb21636c9...fcc6b3e0`
Cycle	cross-version `pancake_v2:WBNB/USDT` → `pancake_v3:WBNB/USDT`
Optimal input	6.21 WBNB
Gross (EVM)	0.00158 BNB (∼$0.93)
Gas	0.00084 BNB (280k gas @ 3 gwei, ∼$0.49)
NET	**+0.00074 BNB (∼$0.43)**
Break-even gas price	5.64 gwei

Table 9: Sandwich funnel over 2,550 sampled blocks (geth-sim15).

Stage	Count	Note
Victim swaps seen	33,525	any V2/V3 `Swap`, any pool
− `skippedUnsupported`	11,695	non-Pancake V3 / Algebra (not yet routed)
− `skippedNoNumeraire`	2,493	pure token/token pools (no BNB-priced side)
− `skippedUnfundable`	1,883	`balanceOf` slot not reproducible
− `belowThreshold`	7,249	below the min-notional screen
− gross-non-positive	8,470	constructed but EVM gross ≤ 0 (slippage gu
Gross-positive (EVM)	**1,735**	EVM-true positive gross, BNB numeraire
Net-positive @ 3 gwei	**1,162**	67.0% of gross-positive (69.0%/1,197 clear ga
Total net	**35.08 BNB (∼$20,200)**	sum of net over the 1,162

Table 10: Sandwich gross-profit distribution.

Statistic	Gross profit (USD)
grossPosSamples (count)	1,735
grossUSD p50	$1.78
grossUSD p90	$31.6
grossUSD p99	$100
grossUSD max (exact)	**$430.50**

Table 11: Sandwich break-even gas-price distribution.

Statistic	Break-even gas price (gwei)
breakevenGwei p50	10
breakevenGwei p90	178
breakevenGwei max (exact)	2,485

Table 12: Sandwich gas-price sensitivity sweep (gross of flash fee).

Gas price (gwei)	Would-be net+ (gross of flash fee)	Share of gross-positive
0	1,735	100.0%
0.1	1,606	92.6%
0.3	1,488	85.8%
1	1,317	75.9%
3 (detector default)	1,197	69.0%

Table 13: Worked example of an ex-post net-positive sandwich.

Field	Value
Block	102,491,896
Victim tx	`0x1f04d512...d5d6486`
Pool	long-tail V2 (`v2_any`), WBNB-numeraire
Direction	WBNB $\rightarrow$ memecoin (victim buys)
Victim input	0.886 WBNB
Optimal frontrun	0.886 WBNB
Gross (EVM)	0.0561 BNB ($\sim$\$32.4)
Gas	0.0009 BNB
Flash fee	0.0008 BNB
NET	**+0.0544 BNB ($\sim$\$31.4)**

Table 14: Backrun vs. sandwich on one instrument (3-gwei gate).

Strategy	Scope	Blocks	Examined	Gross+	Net+ @3gwei	Total net	Med. gross	M
Cross-DEX backrun	deep WBNB/stable hub (12 pools)	3,000	803 cycles	803	**15 (1.9%)**	0.0172 BNB ($\sim$\$10)	\$0.0006	
Sandwich	long-tail WBNB-paired (any-pool)	2,550	33,525 victims	1,735	**1,162 (67.0%)**	35.08 BNB ($\sim$\$20,200)	\$1.78	

Table 15: Realizability of the ex-post sandwich surface over 2,100 sampled blocks (geth-sim16).

Quantity	Value (2,100 blocks)
Ex-post net-positive sandwich opportunities	735
Already captured in-block by a real competitor	**0**
Left on the table (uncaptured)	735
Capture rate	**0.00**
Left-on-table ex-post net (BNB)	30.93

Table 16: Realizability detection funnel and corroboration-failure breakdown.

Funnel stage	Count
Bracket candidates (opposite-dir cross-tx leg pairs, same pool)	1,077
→ passed same-actor gate	79
→ corroborated as a profitable round-trip sandwich	**0**
(corroboration failures: not-flat / hub-negative / below-dust)	74 / 5 / 0

Table 17: Detector recall by injected sandwich structure, and false-positive rate on clean victims.

Injected structure	Recall	FP rate
same-EOA, `Swap`-sender = EOA	0.950	—
same-EOA, sender/beneficiary = contract (integrated-bot pattern)	0.950	—
cross-EOA, shared beneficiary	0.959	—
routed / multi-hop (router as `Swap`-sender)	0.950	—
stable-numeraire hub	0.948	—
thin pool (sub-dust profit)	0.525	—
marginally-flat round trip (>2% Y remainder) — *blind spot*	0.000	—
proceeds swept to a cold address mid-tx — *blind spot*	0.000	—
clean (non-sandwiched) victims	—	**0.000**

Table 18: Model-capacity ladder under identical GroupKFold-by-block CV. CV performance is flat / non-increasing in capacity; XGBoost is the best model on T2 and no deep net beats it.

Model class	T1 curl (CV R^2)	T2 $\log V$ (CV R^2)
Linear (Ridge/Lasso)	0.19	0.65
XGBoost (tree-SOTA)	0.21–0.25	**0.72–0.92**
MLP small (2×64) → large (6×512, 1.6M params)	0.21 → 0.22	0.76 → 0.78
FT-Transformer small → large (≈200k params)	0.15 → 0.21	0.37 → 0.71